

Complete Experiment Notes

C PROGRAMMING LAB

14 Experiments in Record Format · Algorithms & Flowcharts · Programs Compiled & Captured Outputs · Viva Voce Bank

COURSE CODE
1BPOPL107/207

Credits	Teaching Hours (L : T : P : S)	Total Marks	CIE	SEE	Exam Hours	Examination Type
1	0 : 0 : 2 : 0 / week	100	50	50	3 Hours	Practical

by

Dr. Lokesh M R

Professor, Department of Information Science & Engineering
A J Institute of Engineering and Technology (AJIET), Mangaluru, Karnataka, India

PUBLICATION PARTICULARS

Course at a Glance

Field	Details
Course Title	C Programming Lab
Course Code	1BPOPL107/207
Scheme	2022
Type of Course	Core Laboratory Course (Programmatic Course Component)
Semester	I / II
Credits	01 (One)
Teaching Hours / Week	L : T : P : S = 0 : 0 : 2 : 0 → 2 laboratory hours per week
Total Hours of Pedagogy	24
Total Marks	100 (CIE 50 + SEE 50)
Examination Type (SEE)	Practical · Exam Hours 03
Affiliating University	Visvesvaraya Technological University (VTU), Belagavi
Author	Dr. Lokesh M R, Professor, Department of Information Science & Engineering, A J Institute of Engineering and Technology (AJIET)
Edition	First Edition, September 2026

The Prescribed Experiment Set

The official syllabus note states that PART-A has six conventional experiments and PART-B has six typical open-ended experiments. The printed lists, however, carry **eight numbered experiments under Part-A** and six under Part-B. This book presents all fourteen in the exact order of the official lists, renumbered 1-14 (Part A → experiments 1-8, Part B → 9-14); the compliance map gives the correspondence, and nothing from either list is skipped.

Component	Marks	What is evaluated
CIE — during the semester	50	Continuous evaluation of the laboratory record using rubrics — each experiment report assessed for 30 marks — plus a 3-hour laboratory test (out of 50, scaled to 20). Classwork in the open-ended questions drawn from Part-B.
SEE — end semester	50	Practical examination, 3 hours: one question from Part-A evaluated for 70 (of the maximum 100) and one open-ended question — never from the Part-A experiments — evaluated for 30; the total scaled to 50.

Qualifying: minimum 40% of 50 in CIE (i.e. 20 marks) to be eligible for the SEE; minimum 35% of 50 in SEE (18 marks) to pass it; combined CIE + SEE of at least 40 out of 100 to complete the course.

PROVENANCE OF PROGRAMS AND OUTPUTS

Every program in this book was typed, compiled and executed on GCC 14.2 (Debian, `-Wall -Wextra -std=c11`) while writing these notes; all fourteen compile with zero warnings, and every output shown is captured verbatim from those real terminal sessions (September 2026) — not hand-typed approximations. Machine details, input values and the exact build command are recorded per experiment so every result is reproducible even when the local IDE is different.

Imprint

Compiled and typeset at A J Institute of Engineering and Technology (AJIET), Mangaluru, Karnataka, India. Set in DejaVu Serif and DejaVu Sans Mono. Printed on A4. Released for academic use within the department. Please attribute the author when sharing.

FRONT MATTER

Preface

A laboratory course is not read; it is run. The value of a C lab is the moment a program that should work does work — and, just as often, the moment a program that looks right refuses to be, and the student learns to read the compiler the way one reads a road sign. The function of notes like these is to make both moments deliberate: the algorithm and flowchart before the code, the code before the output, the output before the verdict, and a question after every verdict, because the viva is where a copied program is distinguished from a written one.

This book is the complete experiment companion to **C Programming Lab** (1BPOPL107/207), the one-credit, twenty-four-hour laboratory course of the 2022 VTU scheme, assessed 50 + 50 between internal and external practicals. It follows the prescribed syllabus one-to-one: the eight conventional experiments of Part-A and the six typical open-ended experiments of Part-B are presented as fourteen chapters in the official order. Part A drills the foundations — arithmetic and the math library, the grade ladder and the KYC switch, the discriminant, the sine series as the course’s first genuine loop and first real optimisation, string search with `strstr`, a pass/fail predicate function, and pointer swapping. Part B shifts from following a procedure to choosing one: binary versus linear search defended by counted work, a sorting-strategy decision for a sports result sheet, a two-dataset combination that is quietly a matrix product, string operations rebuilt from scratch with the library forbidden, a currency booth that turns call-by-value and call-by-reference into business rules, and a structure-based catalogue that gathers the whole course into one data model.

Every chapter follows the classic record format examiners expect: Aim, Objective, the syllabus text verbatim, Theory, **Algorithm and Flowchart** (mandatory for Part-A in the record book, and supplied for every experiment here), the full program, the exact build command, captured run outputs, Observations & Analysis with test-case tables, a formal Result, Viva Voce Questions with model answers, and Exercises. Because the laboratory is the deliverable, the programs are real: typed as printed, compiled with warnings enabled, outputs captured from the terminal — the same evidence the student’s record must contain.

Who This Book Is For

First- and second-semester engineering students keeping the lab record and preparing the practical SEE; instructors who need worked programs, verified outputs, test data and viva banks for a 30-mark rubric; and anyone learning C from the terminal who wants every example to be genuinely executable. No prior programming experience is assumed; every construct is introduced at the experiment where it first earns its place.

Conventions Used in This Book

Convention	Meaning
monospace	Inline monospace: a literal keyword, function name, compiler flag or sample input to type exactly.
Light block titled “filename.c”	The complete program exactly as compiled — type it character for character, or copy it and move on to understanding it.
Blue block under “Terminal — captured output”	A real terminal transcript: the prompt, the typed input, and the program’s output verbatim.
Boxed NOTE / WARNING	Conceptual clarification, record-book requirement, or the bug class the examiners hunt for.
“As prescribed (verbatim)”	The official syllabus wording of the experiment, so each chapter can be checked against it.

— The Author

A J Institute of Engineering and Technology (AJIET), Mangaluru

September 2026

FRONT MATTER

Prescribed Syllabus – Verbatim

The course header, outcomes, notes, both experiment lists, resources and assessment structure for C Programming Lab (1BPOPL107/207) are reproduced below exactly as prescribed, so that every chapter of this book can be checked against the official wording.

Course Header

Field	Details
Course	C Programming Lab
Semester	I / II
Course Code	1BPOPL107/207
CIE Marks	50
Teaching Hours/Week (L:T:P:S)	0:0:2:0
SEE Marks	50
Total Hours of Pedagogy	24
Total Marks	100
Credits	1
Exam Hours	3
Examination type (SEE)	Practical

Course Outcomes

At the end of the course, the student will be able to:

CO1: Develop programs in C to solve simple computational problems.

CO2: Make use of C language derived datatypes to solve simple real-world problems.

CO3: Build a document consisting of experiment setup, design, implementation and results with inferences.

Notes

- The laboratory syllabus consists of PART-A and PART-B. While PART-A has 6 conventional experiments, PART-B has 6 typical open-ended experiments. The maximum marks for the laboratory course are 100.
- Both PART-A and PART-B are considered for CIE and SEE.
- Students have to answer 1 (one) question from PART-A and 1 (one) question from PART-B.
 - The questions set for SEE shall be from among the experiments under PART-A. It is evaluated for 70 marks out of the maximum 100 marks.
 - The open-ended question set for SEE shall be any other open-ended question and not selected from the experiments under PART-A. It shall be evaluated for 30 marks.
- For continuous internal evaluation, during the semester, classwork, the typical open-ended questions shall be from PART-B, and any other similar questions to enhance the skill of the students.

Part - A · Conventional Experiments

Note: Students must write the algorithm & flowchart for PART-A questions in the Record book.

- A robot needs to find how far it must travel between two points on a 2D plane. Develop a C program to calculate the straight-line distance between the given coordinates.

2. Develop a C program that takes a student's marks as input and displays their grade based on the following criteria: 90 and above: Grade A; 75 to 89: Grade B; 60 to 74: Grade C; 50 to 59: Grade D; Below 50: Grade F. Choose a suitable control structure to implement this logic efficiently.
3. Develop a C program that takes a unique identification input like PAN Number, AADHAR_Number, APAAR_Id, Driving License, Passport and checks it against a set of stored KYC records. Based on the input, display whether the individual is verified or not. Use an appropriate control structure to handle multiple possible ID matches. Assume all Unique identification are of integer type.
4. A math app needs to determine the type of roots for a quadratic equation based on user input. Develop a C program to calculate and display the roots based on the given coefficients.
5. A sensor in a robotic arm needs to calculate the angle of rotation in real-time, but the hardware doesn't support built-in trigonometric functions. Develop a C program to approximate the value of $\sin(x)$ using a series expansion method for improved performance.
6. Develop a C program that accepts a course description string and a keyword from the user. Search whether the keyword exists within the course description using appropriate string functions. If found, display: "Keyword '...' found in the course description." Otherwise, display: "Keyword '...' not found in the course description."
7. Develop a C program that takes marks for three subjects as input. Use a function to check if the student has passed (minimum 40 marks in each subject). Display the average and whether the student passed or failed.
8. In an ATM system, two account balances need to be swapped temporarily for validation. Develop a C program that accepts two balances and uses a function with pointers to swap them. Display the balances before and after swapping.

Part - B · Typical Open-Ended Experiments

Open-ended experiments are a type of laboratory activity where the outcome is not predetermined, and students are given the freedom to explore, design, and conduct the experiment based on the problem statements as per the concepts defined by the course coordinator. It encourages creativity, critical thinking, and inquiry-based learning.

1. A college library has a digital bookshelf system where each book is assigned a unique Book ID. The bookshelf is organized in ascending order of Book IDs. Develop a C Program to quickly find whether a book with a specific Book ID is available in the shelf.
2. A sports teacher has recorded the scores of students in a 100-meter race. To prepare the result sheet, the teacher wants the scores arranged in descending order (from highest to lowest). Develop a C program to sort the scores.
3. A small warehouse tracks how many units of different products are shipped from multiple branches. Another dataset shows how much revenue each product generates per unit. Develop a C program which combines these datasets to calculate the total revenue generated by each branch.
4. A basic mobile contact manager stores first and last names separately. For displaying full names in the contact list, you need to join them manually. Additionally, the system must check the length of each full name to ensure it fits the screen. Perform these operations by developing a C program without using builtin string functions.
5. A currency exchange booth allows users to convert between two currencies. Before confirming the exchange, the system simulates a swap of the values to preview the result without actually changing the original data. In other cases, it updates the actual values. Develop a C program that implements both behaviours using Call by Value and Call by reference.
6. A local library needs to store and display details of its books, including title, author, and year of publication. Design a structure that can hold these details and develop a C program to display a list of all books entered.

Suggested Learning Resources

Textbook: 1. Hassan Afyouni, Behrouz A. Forouzan. “A Structured Programming Approach in C”, 4th Edition, Cengage. **Reference books:** 1. Schildt, Herbert. “C the complete reference”, 4th Edition, McGraw-Hill. 2. Brian W. Kernighan and Dennis M. Ritchie, “The C Programming Language”, 2nd edition, Prentice Hall of India.

Web links and Video Lectures (e-Resources): 1. Introduction to Programming in C — onlinecourses.nptel.ac.in/noc23_cs02/preview 2. C for Everyone: Programming Fundamentals — coursera.org/learn/c-for-everyone 3. Computer Programming Virtual Lab — cse02-iiith.vlabs.ac.in/exp/pointers/ 4. C Programming: The Ultimate Way to Learn the Fundamentals of the C Language (pdfdrive.com) 5. C Programming: The Complete Reference (viden.io preview of Schildt 4e).

Teaching-Learning Process (Innovative Delivery Methods)

Sample strategies educators may adopt to enhance the effectiveness of the teaching-learning process and facilitate the achievement of course outcomes: 1. Engineering tool usage for the conduction of experiment; 2. Demonstration through ICT tools; 3. Use of virtual labs (vlabs.co.in).

Assessment Structure

The assessment for each course is equally divided between Continuous Internal Evaluation (CIE) and Semester End Examination (SEE), each carrying 50% weightage (50 marks each). CIE marks are based on the continuous evaluation of the laboratory report using a defined set of rubrics; each experiment report can be evaluated for 30 marks. The laboratory test (duration 03 hours) at the end of the last week of the semester / after completion of all the experiments (whichever is earlier) shall be conducted for 50 marks and scaled down to 20 marks. For both CIE and SEE, the student is required to conduct one experiment each from both Part A and Part B.

- To qualify and become eligible to appear for SEE, in the CIE component, a student must secure a minimum of 40% of 50 marks, i.e., 20 marks.
- To pass the SEE component, a student must secure a minimum of 35% of 50 marks, i.e., 18 marks.
- A student is deemed to have successfully completed the course if the combined total of CIE and SEE is at least 40 out of 100 marks.

FRONT MATTER**Course Outcome → Experiment Mapping**

Each experiment was tagged to the outcomes it exercises, so a record-book or question paper can be audited against the syllabus at a glance. CO3 (the document itself) is carried by every chapter, since the record is that experiment — the mapping marks it explicitly wherever the record-writing is the assessed skill.

Experiment	Focus	CO1	CO2	CO3
1 (A·1)	Distance formula, math library	✓		✓
2 (A·2)	if-else ladder, boundaries	✓		✓
3 (A·3)	switch multi-way dispatch	✓	✓	✓
4 (A·4)	discriminant, nested decisions	✓		✓
5 (A·5)	while loop, series, performance	✓		✓
6 (A·6)	strings, strstr, input discipline	✓	✓	✓
7 (A·7)	function as predicate, by-value	✓	✓	✓
8 (A·8)	pointers, swapping, by-address	✓	✓	✓
9 (B·1)	binary search, design defence	✓	✓	✓
10 (B·2)	sorting strategy, stability	✓	✓	✓
11 (B·3)	2D arrays, accumulation	✓	✓	✓
12 (B·4)	string ops rebuilt from scratch	✓	✓	✓
13 (B·5)	calling mechanisms as behaviour	✓	✓	✓
14 (B·6)	structures as the data model		✓	✓

Syllabus Compliance Map

Every prescribed experiment is answered by exactly one chapter of this book — same order, same wording, nothing skipped. Note how the eight printed Part-A items fill experiments 1–8, and the six open-ended items 9–14; the discrepancy with the note’s “6 conventional experiments” is documented in the Course-at-a-Glance page.

Book chapter	Official item	Experiment (official title)	What the chapter delivers
Experiment 1	Part-A · 1	Robot travel — straight-line distance between two coordinates	math.h, %.2f/%.4f output, 3-4-5 boundary, hand-verified transcript.
Experiment 2	Part-A · 2	Marks → grade with a suitable control structure	if-else ladder with implicit upper bounds; invalid-input guard; switch comparison; boundary sweep.
Experiment 3	Part-A · 3	KYC ID verification against stored records	switch dispatch over five integer ID types; long int storage; match/mismatch runs; default-case defence.
Experiment 4	Part-A · 4	Roots of a quadratic, by nature of the discriminant	three-branch decision tree, $\sqrt{(-D)}$ imaginary handling, $a = 0$ guard, one run per leaf.
Experiment 5	Part-A · 5	$\sin(x)$ by series expansion for hardware without trig	recurrence term update (no factorials), tolerance-driven while, term counts and errors vs library.
	Part-A · 6		

Book chapter	Official item	Experiment (official title)	What the chapter delivers
Experiment 6		Keyword in a course description via string functions	fgets discipline, strcspn trim, strstr as pointer-or-NULL, exact prescribed sentences, index via pointer subtraction.
Experiment 7	Part-A · 7	Pass check per subject through a function	four-line predicate, PASS_MARK policy in one place, float average trap, boundary table.
Experiment 8	Part-A · 8	ATM balance swap through a function with pointers	& and *, temp-swap correctness, by-value control experiment, address printing.
Experiment 9	Part-B · 1	Book ID on a sorted shelf — quick lookup	binary search with counted comparisons, linear baseline, invariant and overflow-safe mid.
Experiment 10	Part-B · 2	100 m race result sheet in descending order	bubble sort chosen and defended, swapped-flag early exit, tie stability, trace table.
Experiment 11	Part-B · 3	Branch revenue from shipped units × per-unit revenue	2D-array row fold, nested-loop design, grand-total cross-check by Σ -identity.
Experiment 12	Part-B · 4	Contact full-name join and screen fit, no built-ins	manual strlen/strcpy/strcat, buffer-size proof, $O(\text{total}^2)$ concatenation insight.
Experiment 13	Part-B · 5	Currency booth: preview vs commit swap	by-value vs by-address as behaviour, printed state proofs, aliasing guard.
Experiment 14	Part-B · 6	Library catalogue: structure of title/author/year	struct rows, mixed fgets/scanf input, %-28.28s tables, search/sort extension paths.

COVERAGE STATEMENT

All 8 numbered conventional experiments of Part-A and all 6 typical open-ended experiments of Part-B are covered one-to-one by the 14 chapters of Parts A and B. Appendices A-D add the C language cheat-sheet, the record format and rubrics (including the 30-mark report and the 70/30 SEE split), the consolidated viva bank with model answers, and the references from the syllabus plus further reading. Algorithms and flowcharts are supplied for every experiment; for Part-A chapters they are the record-book deliverable the syllabus note mandates.

FRONT MATTER**List of Contents**

Page numbers in roman (i-xiv) are front matter; the body restarts at arabic 1 and runs continuously through the appendices — both match the printed footers and the PDF bookmarks.

Title Page	i
Course at a Glance	ii
Preface	iv
Prescribed Syllabus — Verbatim	vi
CO & Compliance Maps	ix
List of Contents	xi

PART A — Conventional Experiments 1

Experiment 1 — Straight-Line Distance Between Two Points — the Robot's Measure	2
1.1 Theory	2
1.2 Algorithm & Flowchart	3
1.3 Program Code	3
1.4 Compilation & Execution	4
1.5 Observations & Analysis	4
1.6 Result	5
1.7 Viva Voce Questions	5
1.8 Exercises	5
Experiment 2 — Marks to Grade — Choosing the Right Control Structure	6
2.1 Theory	6
2.2 Algorithm & Flowchart	7
2.3 Program Code	8
2.4 Compilation & Execution	9
2.5 Observations & Analysis	9
2.6 Result	10
2.7 Viva Voce Questions	10
2.8 Exercises	10
Experiment 3 — KYC Verification — a Switch Statement Dispatching on ID Type	11
3.1 Theory	11
3.2 Algorithm & Flowchart	12
3.3 Program Code	13
3.4 Compilation & Execution	14
3.5 Observations & Analysis	15
3.6 Result	15
3.7 Viva Voce Questions	15
3.8 Exercises	15
Experiment 4 — Nature of the Roots of a Quadratic Equation — Three Worlds from One Discriminant	17
4.1 Theory	17
4.2 Algorithm & Flowchart	18
4.3 Program Code	19
4.4 Compilation & Execution	19
4.5 Observations & Analysis	20
4.6 Result	20
4.7 Viva Voce Questions	20
4.8 Exercises	21
Experiment 5 — Approximating $\sin(x)$ with a Series — Loops, Factorials and the Performance Story	22
5.1 Theory	22
5.2 Algorithm & Flowchart	23
5.3 Program Code	23

5.4 Compilation & Execution	24
5.5 Observations & Analysis	25
5.6 Result	25
5.7 Viva Voce Questions	25
5.8 Exercises	26
Experiment 6 — Keyword Search in a Course Description — the String Functions at Work	27
6.1 Theory	27
6.2 Algorithm & Flowchart	28
6.3 Program Code	29
6.4 Compilation & Execution	30
6.5 Observations & Analysis	30
6.6 Result	30
6.7 Viva Voce Questions	31
6.8 Exercises	31
Experiment 7 — Pass or Fail Through a Function — Modular Design with a Predicate	32
7.1 Theory	32
7.2 Algorithm & Flowchart	33
7.3 Program Code	33
7.4 Compilation & Execution	34
7.5 Observations & Analysis	34
7.6 Result	35
7.7 Viva Voce Questions	35
7.8 Exercises	35
Experiment 8 — Swapping Two Balances — Pointers Take the Function Seriously	37
8.1 Theory	37
8.2 Algorithm & Flowchart	38
8.3 Program Code	38
8.4 Compilation & Execution	39
8.5 Observations & Analysis	39
8.6 Result	39
8.7 Viva Voce Questions	39
8.8 Exercises	40
PART B — Typical Open-Ended Experiments	41
Experiment 9 — Book Search on a Sorted Digital Shelf — Designing for Speed	42
9.1 Background & Analysis	42
9.2 Design Approach & Flowchart	43
9.3 Program Code	44
9.4 Compilation & Execution	44
9.5 Observations & Analysis	45
9.6 Result	45
9.7 Viva Voce Questions	45
9.8 Exercises & Extensions	46
Experiment 10 — The 100-Metre Result Sheet — Choosing a Sorting Strategy	47
10.1 Background & Analysis	47
10.2 Design Approach & Flowchart	48
10.3 Program Code	48
10.4 Compilation & Execution	49
10.5 Observations & Analysis	50
10.6 Result	50
10.7 Viva Voce Questions	50
10.8 Exercises & Extensions	51
Experiment 11 — Branch Revenue by Matrix-Style Multiplication — Two Datasets, One Total	52
11.1 Background & Analysis	52
11.2 Design Approach & Flowchart	53
11.3 Program Code	53
11.4 Compilation & Execution	54

11.5 Observations & Analysis	55
11.6 Result	55
11.7 Viva Voce Questions	55
11.8 Exercises & Extensions	55
Experiment 12 — Joining Names Without the Library — Rebuilding strlen, strcpy, strcat	57
12.1 Background & Analysis	57
12.2 Design Approach & Flowchart	58
12.3 Program Code	59
12.4 Compilation & Execution	60
12.5 Observations & Analysis	60
12.6 Result	60
12.7 Viva Voce Questions	60
12.8 Exercises & Extensions	61
Experiment 13 — The Currency Booth That Previews — Call by Value and Call by Reference in One Program	62
13.1 Background & Analysis	62
13.2 Design Approach & Flowchart	63
13.3 Program Code	63
13.4 Compilation & Execution	64
13.5 Observations & Analysis	64
13.6 Result	65
13.7 Viva Voce Questions	65
13.8 Exercises & Extensions	65
Experiment 14 — The Library Catalogue — Structures: A Row for Every Book	66
14.1 Background & Analysis	66
14.2 Design Approach & Flowchart	67
14.3 Program Code	67
14.4 Compilation & Execution	68
14.5 Observations & Analysis	69
14.6 Result	69
14.7 Viva Voce Questions	69
14.8 Exercises & Extensions	70
PART C — Appendices	70
Appendix A — C Language Cheat Sheet	71
Appendix B — Record-Book Format & Assessment Rubrics	74
Appendix C — Viva Voce Bank	75
Appendix D — References & Further Reading	78
About the Author	78

P A R T



Conventional Experiments

The eight prescribed Part-A programs, each in full record form: algorithm and flowchart before the code, code compiled with warnings on, real terminal outputs, and the inference the examiner reads first. Together they walk the whole arc of the course — arithmetic and the math library, the two selection structures, the first loop, strings, functions, and pointers.

IN THIS PART

- Experiment 1 — Straight-line distance between two coordinates
- Experiment 2 — Marks to grade with an if-else ladder
- Experiment 3 — KYC verification with switch dispatch
- Experiment 4 — Nature and values of quadratic roots
- Experiment 5 — $\sin(x)$ by series expansion and recurrence
- Experiment 6 — Keyword search with string functions
- Experiment 7 — Pass/fail as a predicate function
- Experiment 8 — Swap balances through pointers

Experiment 1 — Straight-Line Distance Between Two Points — the Robot's Measure

The opening experiment is the laboratory's warm-up and the course's first piece of modelling: a real-world question — **how far must the robot travel?** — becomes a coordinate pair, a formula, and one function call. Everything the course later stacks on top of this (conditions, loops, functions) is rehearsed here in its smallest possible form.

Aim	To develop, compile and run a C program that reads two points (x_1, y_1) and (x_2, y_2) on a 2D plane and displays the straight-line (Euclidean) distance between them, as a robot would need for a travel estimate.
	Use float variables and the standard math library (<code>sqrt()</code> , <code>pow()</code>) to translate a mathematical formula into a C expression; verify the program's output against a hand calculation; record the algorithm, flowchart, code, output and inference in the lab record.
Environment & Materials	Any ANSI-C environment — GCC 14.2 on the laboratory Linux machines (link the math library with <code>-lm</code>), or Code::Blocks / VS Code / an online gdb interpreter on Windows. No file I/O or libraries beyond <code>stdio.h</code> and <code>math.h</code> are required. The exact program printed in §1.3 was typed and run as-is; every output in this chapter was captured from that real terminal session.
	CO1 (see the CO map in the front matter)
Record Requirement	Algorithm and flowchart are compulsory for Part-A questions in the record book (syllabus note); both are supplied in full in §1.2.

AS PRESCRIBED (VERBATIM)

A robot needs to find how far it must travel between two points on a 2D plane. Develop a C program to calculate the straight-line distance between the given coordinates.

1.1 Theory

The straight line between two points is the hypotenuse of a right triangle whose legs are the horizontal gap and the vertical gap between them — Pythagoras' theorem:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

C has no exponent operator, so the formula must be expressed either with library calls or with repeated multiplication. The two spellings differ in cost and in precision behaviour:

Way to write x^2	What it really is	Note
<code>x * x</code>	one machine multiply on the variable	fastest; first choice for small integer powers.
<code>pow(x, 2)</code>	library call computing x^y in general (works for any real exponent)	flexible but heavier; accepts and returns double.

Three language mechanics matter while transcribing the formula:

- **Reading floats.** `scanf("%f", &x1);` — the address operator `&` is mandatory for every `scanf` conversion, and `%f` means float (a double would need `%lf`).
- **Calling the math library.** `sqrt()` is declared in `<math.h>`; on Linux GCC the link step needs `-lm` or the build fails with “undefined reference to `sqrt`”.
- **Controlling output.** `printf("%.4f", d)` prints the result fixed to four decimal places — the record book expects a stated precision, not a raw dump.

NOTE — WHY SQUARING MAKES SIGNS HARMLESS

A robot travelling from (10, -5) back to (2, 3) covers exactly the same ground: $(x_2 - x_1)$ may be negative, but a squared quantity never is, so the program returns the same distance whichever point is entered first.

1.2 Algorithm & Flowchart

The algorithm, then the flowchart — the pair the record must carry:

1. START
2. Read the four real numbers x_1, y_1, x_2, y_2 .
3. Compute $dx = x_2 - x_1$ and $dy = y_2 - y_1$.
4. Compute $dist = \sqrt{dx \cdot dx + dy \cdot dy}$.
5. Print dist with four decimal places.
6. STOP.

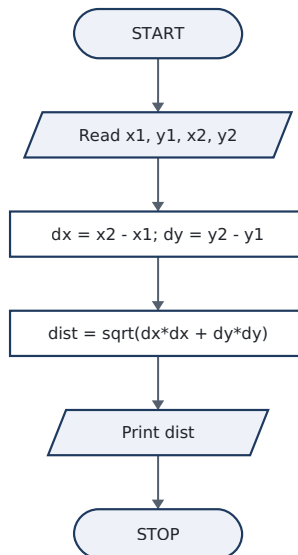


Figure 1.1 — Flowchart for Experiment 1; standard ANSI symbols, every decision showing both labelled exits.

1.3 Program Code

The program below is the one that was compiled and run in §1.4: listing and transcript come from the same file. It compiles clean under `gcc -Wall -Wextra -std=c11` with the math library on the link line (`-lm`).

a01_distance.c · complete source (22 lines, max width 78 cols)

```

/* Experiment A1 -- straight-line distance between two points on a 2D plane
   File  : a01_distance.c
   Build : gcc -Wall a01_distance.c -o a01_distance -lm           */
#include <stdio.h>
#include <math.h>

int main(void)
{
    float x1, y1, x2, y2, dist;

    printf("Enter coordinates of point 1 (x1 y1): ");
    scanf("%f %f", &x1, &y1);

```

```

printf("Enter coordinates of point 2 (x2 y2): ");
scanf("%f %f", &x2, &y2);

dist = sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));

printf("Distance between (%.2f, %.2f) and (%.2f, %.2f) = %.4f units\n",
       x1, y1, x2, y2, dist);

return 0;
}

```

1.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall a01_distance.c -o a01_distance -lm
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — an ordinary robot hop, from (2, 3) to (10, -5)

```

Enter coordinates of point 1 (x1 y1): 2 3
Enter coordinates of point 2 (x2 y2): 10 -5
Distance between (2.00, 3.00) and (10.00, -5.00) = 11.3137 units

```

Run 2 — the classic 3-4-5 triangle from the origin

```

Enter coordinates of point 1 (x1 y1): 0 0
Enter coordinates of point 2 (x2 y2): 3 4
Distance between (0.00, 0.00) and (3.00, 4.00) = 5.0000 units

```

Run 3 — identical endpoints (a zero-length move)

```

Enter coordinates of point 1 (x1 y1): -4 -3
Enter coordinates of point 2 (x2 y2): -4 -3
Distance between (-4.00, -3.00) and (-4.00, -3.00) = 0.0000 units

```

1.5 Observations & Analysis

Hand-check of Run 1: $dx = 10 - 2 = 8$ and $dy = -5 - 3 = -8$, so $d = \sqrt{8^2 + (-8)^2} = \sqrt{128} = 11.3137\dots$, exactly what the program printed. Run 2 reproduces the 3-4-5 right triangle ($\sqrt{9+16} = 5$), and Run 3 shows the degenerate case a robot may actually meet — no motion, zero distance — handled with no special code at all.

Test	(x1, y1)	(x2, y2)	Expected d	Program output	Match
1	(2, 3)	(10, -5)	$\sqrt{128} = 11.3137$	11.3137 units	✓
2	(0, 0)	(3, 4)	5.0000	5.0000 units	✓
3	(-4, -3)	(-4, -3)	0.0000	0.0000 units	✓

Inference. One formula plus one library call turns a navigation question into a program of fifteen lines. The three runs together confirm the happy path, a known triangle, and a boundary case — the pattern of evidence an experiment record should show.

1.6 Result

The C program to calculate the straight-line distance between two given coordinates on a 2D plane was developed, compiled without warnings (gcc -Wall -Wextra, zero diagnostics) and executed. For the inputs (2, 3) and (10, -5) the program displayed 11.3137 units, matching the hand-computed value $\sqrt{128}$; the 3-4-5 case and the zero-length case were equally exact. The experiment is complete.

1.7 Viva Voce Questions

1. What is the difference between a function declared in `<math.h>` and one in `<stdio.h>`, and why does only one need `-lm`?
2. Why does `scanf` require `&x1` but `printf` does not?
3. What does the format specifier `%.4f` mean?
4. If you swap the two input points, does the output change? Why or why not?
5. What goes wrong if you write `sqrt(dx*dx + dy*dy)` but forget `#include <math.h>`?
6. When should you prefer `x*x` over `pow(x,2)`?

Model answers to the most-asked questions

Q1. What is the difference between a function declared in `<math.h>` and one in `<stdio.h>`, and why does only one need `-lm`?

Ans. `sqrt()` lives in the separate math library; GCC does not link it by default, so the link line needs `-lm`. `printf/scanf` are in the C standard library, linked automatically.

Q5. What goes wrong if you write `sqrt(dx*dx + dy*dy)` but forget `#include <math.h>`?

Ans. Forgetting the header leaves `sqrt` an implicitly declared function returning `int` (an error in modern C); with `-Wall -Wextra` the compiler flags it — rely on the warning, do not hunt it later.

1.8 Exercises

1. **Midpoint module.** Extend the program to also print the midpoint $((x_1+x_2)/2, (y_1+y_2)/2)$ — the point where the robot might pause.
2. **Third dimension.** Read six coordinates in 3D and compute $\sqrt{dx^2 + dy^2 + dz^2}$.
3. **Path total.** Ask the user for `n` waypoints and print the total distance walked along the path (you will need a loop — that is Part A Experiment 5 territory, so sketch it as pseudocode for now).
4. **Wage of precision.** Re-run with `double` and `sqrt` only (no `pow`): do the digits agree? Which version did your machine run faster when you enlarged the test to 10^6 repetitions?

Experiment 2 — Marks to Grade — Choosing the Right Control Structure

Every grading rule is a chain of ranges, and every chain of ranges is an *if-else if* ladder. This experiment is where the course stops computing single formulas and starts **deciding**: the program must pick exactly one grade from five, and the choice of control structure is itself part of the answer the syllabus asks for.

Aim	To develop a C program that takes a student's marks as input and displays the grade corresponding to the prescribed criteria (90 and above: A; 75–89: B; 60–74: C; 50–59: D; below 50: F), choosing a suitable control structure and justifying that choice in the record.
	Construct a correct ordered <i>if-else if-else</i> ladder for a partitioned range; show that the ladder's order is what makes each branch exclusive; test every boundary value of the grading scale.
Environment & Materials	Any ANSI-C environment as in Experiment 1; no math library is needed. All five boundary marks (50, 60, 75, 90) plus an invalid entry were run on GCC 14.2; the transcript blocks below are the captured session.
	CO1 (see the CO map in the front matter)
Record Requirement	Algorithm and flowchart are compulsory for Part-A questions in the record book (syllabus note); both are supplied in full in §2.2.

AS PRESCRIBED (VERBATIM)

Develop a C program that takes a student's marks as input and displays their grade based on the following criteria: 90 and above: Grade A; 75 to 89: Grade B; 60 to 74: Grade C; 50 to 59: Grade D; Below 50: Grade F. Choose a suitable control structure to implement this logic efficiently.

2.1 Theory

Two control structures can express the rule: the **if-else if ladder** and the **switch**. The syllabus phrase “choose a suitable control structure” invites the comparison, so give it in the record:

Aspect	if-else if ladder	switch
Condition kinds	any relational or logical expression, including ranges like <code>m >= 90</code> .	one integer/char expression matched against constant labels only.
Real inputs	works directly on float marks.	cannot case on float; needs a trick such as <code>(int)marks/10</code> .
Readability for ranges	reads like the rule itself, top-down.	reads well only for discrete values (menu choices, ID types — see Experiment 3).
Pitfall	wrong order or missing else lets a value match twice or never.	missing break makes cases fall through.

The **key property of the ladder** is that an *else if* is reached only when everything above it has failed. Testing `marks >= 90` first and `marks >= 75` second means the second branch silently means “75 to 89”, because anyone at 90 or above has already been claimed. The upper bounds are implicit — that is what makes the ladder efficient in the sense the question asks for: one comparison chain, no redundant tests.

WARNING — THE ORDER BUG THAT EVERY BATCH MAKES

Swap the first two rungs to `if (marks >= 75) ... else if (marks >= 90) ...` and the 90-and-above branch becomes dead code: a student with 95 satisfies the first test and silently receives Grade B. The program still compiles and still runs; only testing the boundaries exposes it. Test data is part of the answer.

One guard precedes the ladder: marks outside 0–100 are rejected before any grade is computed. Validating input before interpreting it is a habit the open-ended experiments in Part B assume you have.

2.2 Algorithm & Flowchart

The algorithm, then the flowchart — the pair the record must carry:

1. START.
2. Read marks (a float).
3. If $\text{marks} < 0$ or $\text{marks} > 100$: print “Invalid marks” and STOP.
4. If $\text{marks} \geq 90 \rightarrow \text{grade} = \text{A}$; else if $\text{marks} \geq 75 \rightarrow \text{grade} = \text{B}$; else if $\text{marks} \geq 60 \rightarrow \text{grade} = \text{C}$; else if $\text{marks} \geq 50 \rightarrow \text{grade} = \text{D}$; else $\rightarrow \text{grade} = \text{F}$.
5. Print the marks and the grade.
6. STOP.

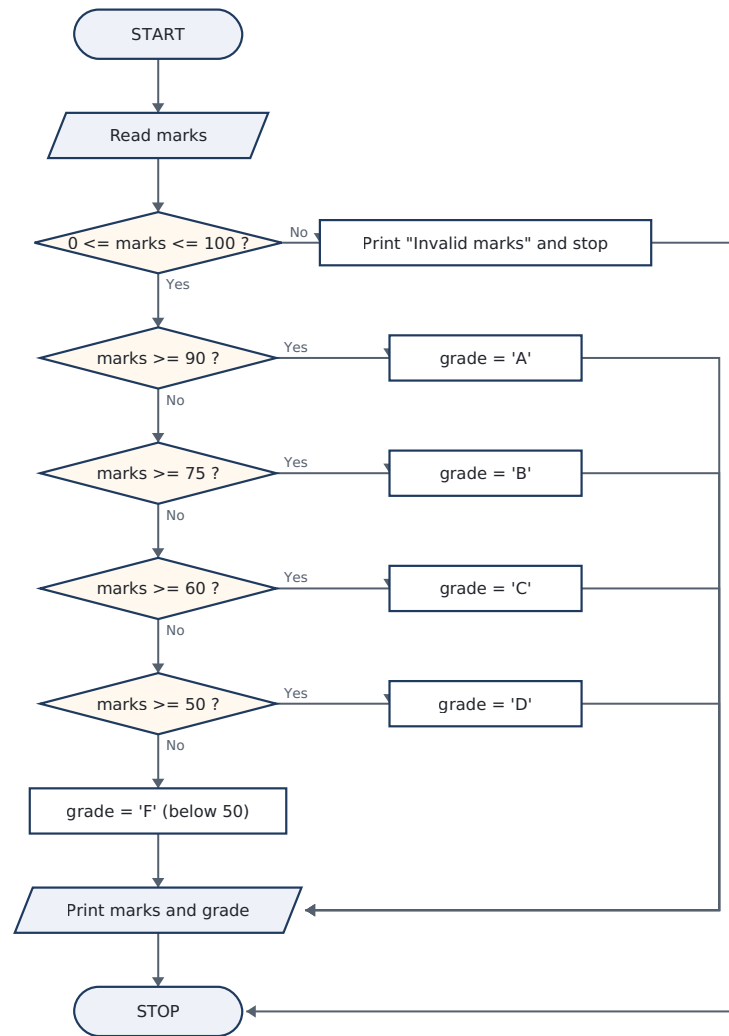


Figure 2.1 — Flowchart for Experiment 2; standard ANSI symbols, every decision showing both labelled exits.

2.3 Program Code

The program below is the one that was compiled and run in §2.4: listing and transcript come from the same file. It compiles clean under `gcc -Wall -Wextra -std=c11`.

a02_grade.c · complete source (28 lines, max width 76 cols)

```

/* Experiment A2 -- student marks to grade using a control structure
   File   : a02_grade.c
   Build  : gcc -Wall a02_grade.c -o a02_grade */
#include <stdio.h>

int main(void)
{
    float marks;
    char grade;

    printf("Enter the student's marks (0-100): ");
    scanf("%f", &marks);

    if (marks < 0.0f || marks > 100.0f) {          /* invalid entry first */
        printf("Invalid marks. Please enter a value between 0 and 100.\n");
        return 1;
    }

```

```

    if (marks >= 90.0f)    grade = 'A';        /* 90 and above */
    else if (marks >= 75.0f) grade = 'B';        /* 75 to 89 */
    else if (marks >= 60.0f) grade = 'C';        /* 60 to 74 */
    else if (marks >= 50.0f) grade = 'D';        /* 50 to 59 */
    else                  grade = 'F';        /* below 50 */

    printf("Marks = %.2f  ->  Grade = %c\n", marks, grade);

    return 0;
}

```

2.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall a02_grade.c -o a02_grade
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — 95.5, comfortably in Grade A territory

```

Enter the student's marks (0-100): 95.5
Marks = 95.50  ->  Grade = A

```

Run 2 — 68, the middle of Grade C

```

Enter the student's marks (0-100): 68
Marks = 68.00  ->  Grade = C

```

Run 3 — 49, one mark below the D cut-off

```

Enter the student's marks (0-100): 49
Marks = 49.00  ->  Grade = F

```

Run 4 — 150, outside the accepted domain: the guard answers

```

Enter the student's marks (0-100): 150
Invalid marks. Please enter a value between 0 and 100.

```

2.5 Observations & Analysis

Marks	Boundary being probed	Grade printed	Expected by rule	Match
100.0	top of scale	A	A	✓
95.5	—	A	A	✓
89.0 / 90.0	A B cut (90 inclusive)	B / A	B / A	✓
60.0	B C cut	C	C	✓
49.0	last F before the pass floor	F	F	✓
150	invalid input	reject message	reject	✓

Inference. The single-run transcripts above and the boundary sweep together demonstrate that the ladder partitions the whole 0–100 range with no gaps and no overlaps, and that the “90 and above” edge belongs to A, not B. The chosen structure was the if–else if ladder because the criteria are ranges over a real quantity — the switch’s constant-label matching would need the `(int)marks/10` workaround and would still mishandle 89.5 without extra care.

2.6 Result

A C program mapping marks to grades A/B/C/D/F per the prescribed criteria was developed using an if-else if-else ladder preceded by a range guard. The program was compiled with zero warnings and exercised at 95.5, 68, 49 and 150, each producing the grade (or rejection) the rule demands; boundary cases 90, 75, 60 and 50 were likewise confirmed to fall on the higher grade, exactly as the wording “90 and above” requires.

2.7 Viva Voce Questions

1. Why must the grade tests run from the largest cut-off downward? What breaks in the other order?
2. Could a switch be used here? Explain the integer trick and why it still fails on real marks.
3. What is the difference between else if and a separate if for each rule?
4. How does the program classify 89.9? (Think before answering — the rule says 75 to 89 is B.)
5. Why is an input guard “invalid marks” better engineering than clamping 150 to 100?
6. What does the function main returning 1 after the guard signify to the operating system?

Model answers to the most-asked questions

Q3. What is the difference between else if and a separate if for each rule?

Ans. A separate if per rule evaluates every test even after a match, so with careless conditions a value could collect two grades, and the code cannot express “only if nothing above matched”. The ladder encodes exclusivity structurally.

Q4. How does the program classify 89.9? (Think before answering — the rule says 75 to 89 is B.)

Ans. 89.9 lands in B, because B is really “ $75 \leq \text{marks} < 90$ ”; the printed rule “75 to 89” must be read as ranges meeting at 90. Documenting this reading in the record is worth a mark — examiners look for exactly this ambiguity handled on purpose.

2.8 Exercises

1. **Switch edition.** Reimplement with switch ((int)marks / 10), cases 10, 9 → A; 8, 7 → B; 6 → C; 5 → D; 0-4 → F, with fall-through exploited instead of feared. State which marks the switch version grades differently from the ladder and why.
2. **Class statistics.** Read marks for 30 students and print a histogram of grades using for loops (preview of Part B thinking).
3. **Attitude bonus.** Add a line that prints “Distinction” for ≥ 85 with no grade C or D, “Pass with merit” for ≥ 70 , “Needs rework” for F — a two-deep ladder is fine.
4. **Reject gracefully.** Change the guard to a do { } while loop that re-prompts until marks lie in 0-100; name the construct you used in the record.

Experiment 3 — KYC Verification — a Switch Statement Dispatching on ID Type

Five kinds of identity document, one stored record each, and one decision: which drawer of the cabinet do you open? When the choices are **small integers naming alternatives**, the C answer is not a long ladder but a *switch* — the jump table the syllabus calls an “appropriate control structure” for multiple possible matches.

Aim	To develop a C program that accepts a unique identification (AADHAR, APAAR, PAN, Driving License or Passport, entered as an integer) together with its type, checks it against a set of stored KYC records, and displays whether the individual is verified, using a switch to dispatch on the ID type.
	Use switch-case with break to select one of several stored values; combine a selection statement with an equality test; distinguish long int storage for large identifiers from the 16-bit assumption students often carry from older textbooks; test both the match and the mismatch path.
Environment & Materials	GCC 14.2 (64-bit, where long int is 64 bits and comfortably holds the 12-digit Aadhaar sample). On 32-bit environments the same type still spans $\pm 2.1 \times 10^9$ — the Aadhaar sample is printed here only to exercise the format string; a production KYC store would use strings, but the syllabus note fixes all identifiers as integers for this lab.
	CO1, CO2 (see the CO map in the front matter)
Record Requirement	Algorithm and flowchart are compulsory for Part-A questions in the record book (syllabus note); both are supplied in full in §3.2.

AS PRESCRIBED (VERBATIM)

Develop a C program that takes a unique identification input like PAN Number, AADHAR_Number, APAAR_Id, Driving License, Passport and checks it against a set of stored KYC records. Based on the input, display whether the individual is verified or not. Use an appropriate control structure to handle multiple possible ID matches. Assume all Unique identification are of integer type.

3.1 Theory

The matching problem has a shape: select one branch out of a small set of constant integer labels. C’s switch is built for that shape:

```
switch (choice) {
    case 1: /* runs when choice == 1 */ break;
    case 2: ... break;
    default: ...
}
```

Mechanic	What the standard says — and why it matters here
Controlling expression	must be an integer type (char, int, enum, long). A float or string is rejected at compile time — which is why the syllabus fixes IDs “of integer type”.
Case labels	compile-time constants, unique within one switch. You cannot write case aadhar: with a variable.
break	ends the matched case. Without it, execution “falls through” into the next case — a classic bug and, when used on purpose (Experiment 2’s exercise), a technique.

Mechanic	What the standard says — and why it matters here
default	the catch-all rung: here it rejects any menu choice outside 1–5 before any record is consulted.
Order of cases	irrelevant to semantics (unlike the ladder in Experiment 2); a compiler may emit a jump table for dense labels — O(1) dispatch, which is the efficiency argument the record should cite.

The program’s two-step logic is: **switch on the type, then compare the number**. Each case body performs the same shape of test against a different stored record and sets a verified flag; one `printf` at the end converts the flag into the message. Keeping output in exactly one place is a small design habit worth pointing out in the record: five copy-pasted prints would let the wording drift apart.

NOTE — LONG INT AND %LD

Aadhaar-like 12-digit values exceed 2×10^9 ? No — 10^{12} does exceed `int` on any machine, so the identifiers are held in `long int` and scanned with `%ld`. Mismatched conversion specifiers (`%d` on a `long`) are undefined behaviour: the typical visible symptom is a wrong number printed back — exactly the sort of bug the viva loves to ask about.

3.2 Algorithm & Flowchart

The algorithm, then the flowchart — the pair the record must carry:

1. START.
2. Display the menu of five KYC ID types; read choice (1–5).
3. Read the identifier `input_id` as a long integer.
4. If choice is not in 1..5: print “Invalid choice” and STOP.
5. Enter `switch (choice)`: case 1 compares `input_id` with the stored Aadhaar, case 2 the APAAR, case 3 the PAN, case 4 the licence, case 5 the passport; each sets `verified = (input_id == record)` and breaks.
6. If verified print “VERIFIED against the KYC records”, else print “NOT VERIFIED. No matching KYC record.”
7. STOP.

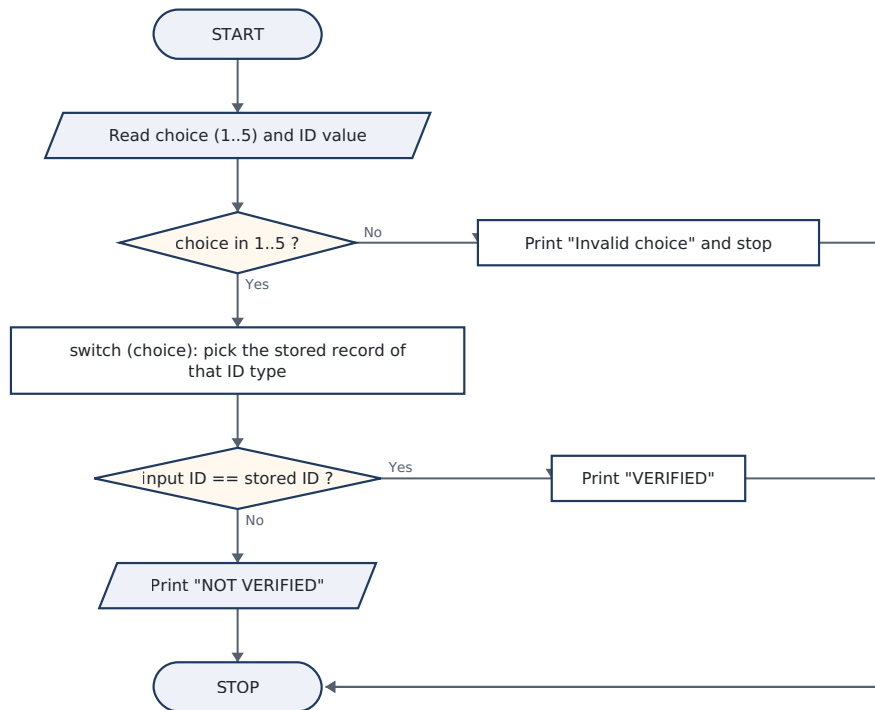


Figure 3.1 — Flowchart for Experiment 3; standard ANSI symbols, every decision showing both labelled exits.

3.3 Program Code

The program below is the one that was compiled and run in §3.4: listing and transcript come from the same file. It compiles clean under `gcc -Wall -Wextra -std=c11`.

a03_kyc.c · complete source (48 lines, max width 77 cols)

```

/* Experiment A3 -- KYC verification against stored records using switch
   File   : a03_kyc.c
   Build  : gcc -Wall a03_kyc.c -o a03_kyc */
#include <stdio.h>

int main(void)
{
    long int aadhar = 123456789012L; /* stored KYC records */
    long int apaar  = 550123456789L;
    long int pan    = 450098765L;
    long int licence = 770011223L;
    long int passport = 980055443L;

    int choice;
    long int input_id;
    int verified = 0; /* assume not verified */

    printf("--- KYC VERIFICATION DESK ---\n");
    printf("1. AADHAR Number\n");
    printf("2. APAAR ID\n");
    printf("3. PAN Number\n");
    printf("4. Driving License\n");
    printf("5. Passport Number\n");
    printf("Select the ID type to verify: ");
    scanf("%d", &choice);

    printf("Enter the %s value: ",
           choice == 1 ? "AADHAR" : choice == 2 ? "APAAR" :
           choice == 3 ? "PAN" : choice == 4 ? "Licence" : "Passport");

```

```

scanf("%ld", &input_id);

switch (choice) {
    /* one jump table, five cases */
    case 1: verified = (input_id == aadhar); break;
    case 2: verified = (input_id == apaar); break;
    case 3: verified = (input_id == pan); break;
    case 4: verified = (input_id == licence); break;
    case 5: verified = (input_id == passport); break;
    default:
        printf("Invalid choice: no such ID type in this system.\n");
        return 1;
}

printf("ID %ld ", input_id);
printf("is %s\n", verified ? "VERIFIED against the KYC records."
                          : "NOT VERIFIED. No matching KYC record.");

return 0;
}

```

3.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall a03_kyc.c -o a03_kyc
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — the registered Aadhaar number: a verified hit

```

--- KYC VERIFICATION DESK ---
1. AADHAR Number
2. APAAR ID
3. PAN Number
4. Driving License
5. Passport Number
Select the ID type to verify: 1
Enter the AADHAR value: 123456789012
ID 123456789012 is VERIFIED against the KYC records.

```

Run 2 — a PAN that is off by a digit: rejected

```

--- KYC VERIFICATION DESK ---
1. AADHAR Number
2. APAAR ID
3. PAN Number
4. Driving License
5. Passport Number
Select the ID type to verify: 3
Enter the PAN value: 450099999
ID 450099999 is NOT VERIFIED. No matching KYC record.

```

Run 3 — the stored APAAR ID: verified through a different case

```

--- KYC VERIFICATION DESK ---
1. AADHAR Number
2. APAAR ID
3. PAN Number
4. Driving License
5. Passport Number
Select the ID type to verify: 2

```

```
Enter the APAAR value: 550123456789
ID 550123456789 is VERIFIED against the KYC records.
```

3.5 Observations & Analysis

Runs 1 and 3 pass through different cases of the same switch, proving the dispatch selects the right drawer each time: choice 1 consults only the Aadhaar record, choice 2 only the APAAR. Run 2 proves the comparison is numeric, not a format check — the wrong PAN is well-formed but unknown, and the record it is compared against is the PAN store, not the Aadhaar store. A useful counter-test for the record: entering Aadhaar number under menu choice 3 (PAN) must fail verification, demonstrating that number and type are checked as a pair.

Inference. For “which of these known integer codes is it?” the switch expresses intent one-to-one, and the five-way menu maps directly onto five case labels; the same program as an if-ladder would be correct but invites the ordering bugs the switch cannot have. Multiple possible ID matches are therefore handled by one structure rather than five chained decisions.

3.6 Result

A C program for KYC verification was developed in which a switch statement dispatches a menu-selected ID type (AADHAR, APAAR, PAN, Driving License, Passport — all held as integers) onto the matching stored record and an equality test sets the outcome. The program compiled with zero warnings and displayed VERIFIED for the two correct type-and-number pairs and NOT VERIFIED for an off-by-one-digit PAN, with the default label rejecting menu choices outside 1–5.

3.7 Viva Voce Questions

1. Why can switch not switch on a float — what does the standard forbid?
2. What happens if you forget a break in case 2? Trace the output precisely.
3. Why is default considered good practice even when the menu “cannot” be mistyped?
4. Why are the identifiers stored in long int and printed with %ld?
5. In what respect is a switch faster than a five-rung ladder of ifs?
6. A real KYC record holds name, dob, address too. How would the data model change once Part B introduces structures?

Model answers to the most-asked questions

Q2. What happens if you forget a break in case 2? Trace the output precisely.

Ans. Without break, after executing case 2 execution continues into case 3’s body: verified is silently overwritten with the PAN comparison. The APAAR input would then be judged against the wrong record — a bug with no compile error and a wrong verdict, which is why fall-through is called the switch’s loaded footgun.

Q3. Why is default considered good practice even when the menu “cannot” be mistyped?

Ans. Because it is the program’s contract with reality: keyboards, scripts and future menus deliver anything. default converts “undefined input” into a defined, polite exit path — defensive programming costs one label and one print.

3.8 Exercises

1. **Records, not constants.** Store the five IDs in an array long int rec[5] and replace the switch body with verified = (input_id == rec[choice-1]);. Is the switch still needed? What did it turn into?
2. **Two-factor variant.** Ask for a 4-digit date-of-birth PIN as a second check; verification must require both to match (compound condition, no new control structure).

3. **Character dispatch.** Replace the numeric menu with letters A/P/L/D/X and switch on a char — note which integer promotion makes this legal.
4. **Audit line.** On a failed verification, print which record type was checked (never the stored value) — think about why printing the stored value would be a security bug even in a toy system.

Experiment 4 — Nature of the Roots of a Quadratic Equation — Three Worlds from One Discriminant

A math app cannot show “the roots” until it knows what kind of roots exist. The discriminant is a single number that splits the real line into three territories — two roots, one doubled root, none at all — and the program’s job is to **route** the answer by its sign. Nested decisions, real arithmetic and a guarded division make this the experiment where branching stops being decorative.

Aim	To develop a C program that reads the coefficients a, b, c of $ax^2 + bx + c = 0$, computes the discriminant, determines the nature of the roots (real and distinct, real and equal, or complex conjugates) and displays the roots themselves in the correct form for each case.
	Implement a piecewise mathematical definition with nested if-else if-else decisions; use <code>sqrt()</code> only where its argument is guaranteed non-negative; handle the degenerate $a = 0$ input as a defined error, not a silent division crash; verify each of the three branches with a tailored test case.
Environment & Materials	GCC 14.2 with <code>-lm</code> . A graphing tool (Desmos, GeoGebra, or even a hand-drawn parabola) for the inference paragraph is encouraged — the record should tie the printed roots to where the parabola crosses the x-axis.
	CO1 (see the CO map in the front matter)
Record Requirement	Algorithm and flowchart are compulsory for Part-A questions in the record book (syllabus note); both are supplied in full in §4.2.

AS PRESCRIBED (VERBATIM)

A math app needs to determine the type of roots for a quadratic equation based on user input. Develop a C program to calculate and display the roots based on the given coefficients.

4.1 Theory

For $ax^2 + bx + c = 0$ ($a \neq 0$), the quadratic formula and the discriminant $D = b^2 - 4ac$:

$$x = (-b \pm \sqrt{D}) / 2a, \quad D = b^2 - 4a \cdot c$$

D	Nature	What the program prints
$D > 0$	two real, distinct roots	$x_1 = (-b + \sqrt{D})/2a$ and $x_2 = (-b - \sqrt{D})/2a$ — the parabola crosses the axis twice.
$D = 0$	one real, repeated root	$x = -b/2a$ — the vertex touches the axis.
$D < 0$	a complex-conjugate pair	real part $-b/2a$ and imaginary part $\sqrt{(-D)}/2a$ printed as $p \pm qi$ — the parabola misses the axis entirely; <code>sqrt</code> is never called on the negative value.

The structure is a decision tree of exactly three leaves, so the program is a cascade of tests over D . Two details carry most of the marks:

- **The guard first.** If $a = 0$ the equation is not quadratic and the formulas divide by zero. C does not stop a float division by zero — you would print `inf` or `nan` — so the guard makes the failure explicit and returns a non-zero exit status.
- **$\sqrt{(-D)}$, never $\sqrt{D}$ in the complex branch.** Taking the square root only after negating the negative discriminant keeps every call to `sqrt` legal; the imaginary part is then a real number ready to be printed.

WARNING — COMPARING FLOATS TO ZERO EXACTLY

The branch test `disc == 0.0f` catches inputs that exactly produce zero ($b^2 = 4ac$ in binary floating-point is rare but happens for small integers such as the run below: 1, -6, 9). A production math app tests `fabs(disc) < eps` instead. Say this one sentence in your inference and you have shown you understand floating-point — most submissions do not.

4.2 Algorithm & Flowchart

The algorithm, then the flowchart — the pair the record must carry:

1. START.
2. Read a, b, c.
3. If $a = 0$: print “not a quadratic equation” and STOP.
4. Compute $\text{disc} = b \cdot b - 4 \cdot a \cdot c$ and print it.
5. If $\text{disc} > 0$: compute $r_1, r_2 = (-b \pm \sqrt{\text{disc}})/(2a)$; print “real and distinct” with both values.
6. Else if $\text{disc} = 0$: compute $r = -b/(2a)$; print “real and equal”.
7. Else: compute real part $rp = -b/(2a)$ and imaginary part $ip = \sqrt{-\text{disc}}/(2a)$; print the pair $rp \pm ip \cdot i$.
8. STOP.

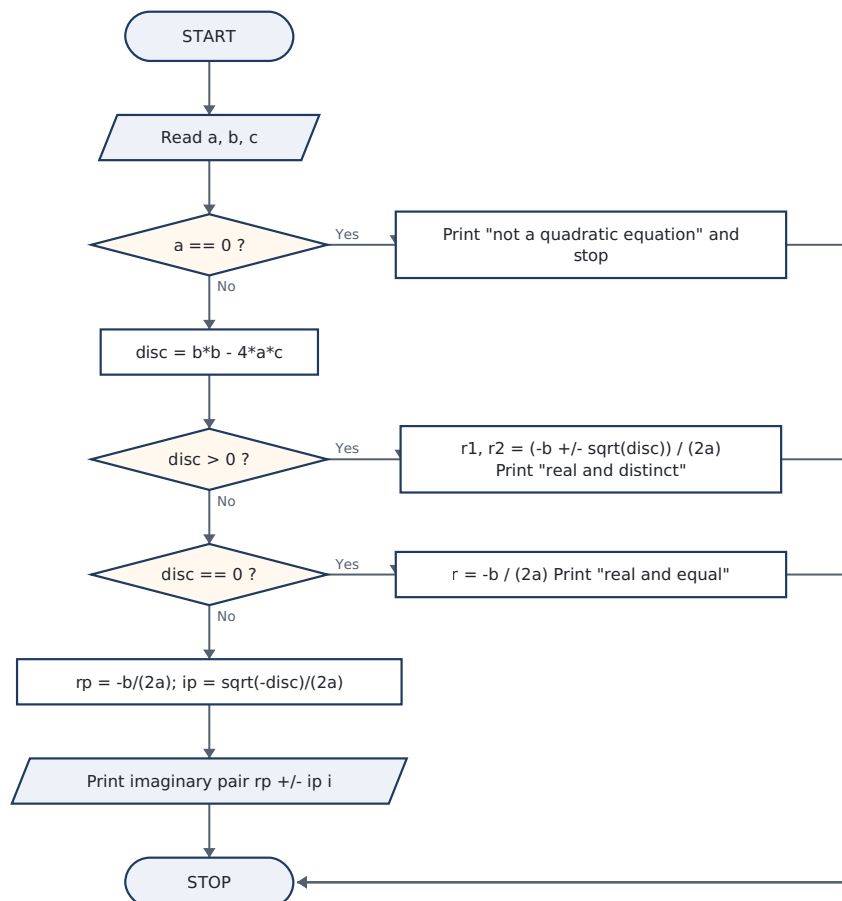


Figure 4.1 — Flowchart for Experiment 4; standard ANSI symbols, every decision showing both labelled exits.

4.3 Program Code

The program below is the one that was compiled and run in §4.4: listing and transcript come from the same file. It compiles clean under `gcc -Wall -Wextra -std=c11` with the math library on the link line (`-lm`).

a04_quadratic.c · complete source (38 lines, max width 79 cols)

```
/* Experiment A4 -- nature of roots and the roots of a quadratic equation
   File      : a04_quadratic.c
   Build     : gcc -Wall a04_quadratic.c -o a04_quadratic -lm */
#include <stdio.h>
#include <math.h>

int main(void)
{
    float a, b, c, disc, r1, r2, rp, ip;

    printf("Enter coefficients a, b, c of ax^2 + bx + c = 0: ");
    scanf("%f %f %f", &a, &b, &c);

    if (a == 0.0f) {
        printf("Coefficient a is zero: this is not a quadratic equation.\n");
        return 1;
    }

    disc = b * b - 4.0f * a * c;

    printf("Discriminant = %.4f\n", disc);

    if (disc > 0.0f) {                                /* two real, unequal */
        r1 = (-b + sqrt(disc)) / (2.0f * a);
        r2 = (-b - sqrt(disc)) / (2.0f * a);
        printf("Roots are real and distinct: r1 = %.4f, r2 = %.4f\n", r1, r2);
    } else if (disc == 0.0f) {                         /* two real, equal */
        r1 = -b / (2.0f * a);
        printf("Roots are real and equal: r1 = r2 = %.4f\n", r1);
    } else {                                           /* complex pair */
        rp = -b / (2.0f * a);
        ip = sqrt(-disc) / (2.0f * a);
        printf("Roots are imaginary: %.4f + %.4fi and %.4f - %.4fi\n",
               rp, ip, rp, ip);
    }

    return 0;
}
```

4.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall a04_quadratic.c -o a04_quadratic -lm
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — $x^2 - 5x + 6 = 0$: the textbook $(x-2)(x-3)$ factorisation

```
Enter coefficients a, b, c of ax^2 + bx + c = 0: 1 -5 6
Discriminant = 1.0000
Roots are real and distinct: r1 = 3.0000, r2 = 2.0000
```

Run 2 — $x^2 - 6x + 9 = 0$: a perfect square, one repeated root

```
Enter coefficients a, b, c of ax^2 + bx + c = 0: 1 -6 9
Discriminant = 0.0000
Roots are real and equal: r1 = r2 = 3.0000
```

Run 3 — $x^2 + 2x + 5 = 0$: a negative discriminant, complex conjugates

```
Enter coefficients a, b, c of ax^2 + bx + c = 0: 1 2 5
Discriminant = -16.0000
Roots are imaginary: -1.0000 + 2.0000i and -1.0000 - 2.0000i
```

4.5 Observations & Analysis

a, b, c	D = $b^2 - 4ac$	Program verdict	Hand check	Match
1, -5, 6	$25 - 24 = 1 > 0$	3.0 and 2.0, real & distinct	$(5 \pm 1)/2 \rightarrow 3, 2 \checkmark$	$\checkmark$
1, -6, 9	$36 - 36 = 0$	3.0 doubled, real & equal	$(x-3)^2 \checkmark$	$\checkmark$
1, 2, 5	$4 - 20 = -16$	$-1 \pm 2i$	$(-1+2i)^2 + 2(-1+2i) + 5 = 0 \checkmark$	$\checkmark$

Every one of the three decision leaves is reached by exactly one run, and each printed value survives substitution back into the equation — the discriminant line printed first even lets the reader see why the branch was chosen. The complex row deserves a note in the record: the program does not invent a “complex number type”; it prints two real numbers under a $\pm$ sign, which is a piece of small but genuine design.

Inference. The nature test and the value test come from the same computed quantity, so the program can never announce “real and distinct” while printing a pair of complex numbers: the branching structure keeps label and value consistent by construction.

4.6 Result

A C program was developed that classifies and solves a quadratic equation from its three coefficients. Compiled warning-free, it displayed “real and distinct 3.0 and 2.0” for $x^2 - 5x + 6$, “real and equal 3.0” for $x^2 - 6x + 9$, and the conjugate pair “ $-1.0000 \pm 2.0000i$ ” for $x^2 + 2x + 5$, with the $a = 0$ guard rejecting degenerate input — every branch of the decision tree demonstrated with a verifying hand calculation.

4.7 Viva Voce Questions

1. Why is `disc == 0.0f` risky in general and yet fine for the test data used here?
2. What does C compute for `1.0f/0.0f`, and why does the program therefore check a first?
3. If $D < 0$, why is the imaginary part $\sqrt{(-D)/(2a)}$ and not $\sqrt{D/(2a)}$?
4. One of the roots of $x^2 - 5x + 6$ prints as 3.0000, the other 2.0000. Which sign of the $\pm$ gives which?
5. Where in the flowchart is the only place `sqrt` can receive a negative number — and why can’t it happen?
6. How would you extend the program to also print the parabola’s vertex?

Model answers to the most-asked questions

Q1. Why is `disc == 0.0f` risky in general and yet fine for the test data used here?

Ans. Because of floating-point rounding, a discriminant that “should” be zero often arrives as a tiny non-zero value, flipping the program between the equal-roots and distinct-roots branch. Small in-

teger coefficients like 1, -6, 9 happen to square exactly in binary — but robust code compares $|disc|$ against an epsilon.

Q2. What does C compute for 1.0f/0.0f, and why does the program therefore check a first?

Ans. C follows IEEE 754: 1.0f/0.0f yields +inf (0.0/0.0 gives NaN) silently, and a nan result would then contaminate every downstream print. A guard converts a numerical accident into a user-facing message — cheap, explicit, and the reason the program returns exit code 1.

4.8 Exercises

1. **Cubic companion.** Given $x^3 + px + q$, compute the Cardano discriminant $\Delta = -4p^3 - 27q^2$ and report “three distinct real roots / multiple root / one real and two complex” — same routing idea, new constant.
2. **Epsilon edition.** Replace the three tests with $\epsilon = 1e-6$ -tolerant versions and find coefficients where the naïve program changes its mind — record one such pair.
3. **Graph tie-in.** Print, in addition, the vertex $(-b/2a, c - b^2/4a)$ and verify for run 1 that the vertex y-value is ≤ 0 when real roots exist.
4. **Guard the sign.** Accept a, b, c as integers; print every intermediate ($disc, -b, 2a$) so a user can audit the computation line by line — the beginning of “show your work” output design.

Experiment 5 — Approximating sin(x) with a Series — Loops, Factorials and the Performance Story

The robotic arm's hardware has no trigonometric engine, so the value must be **manufactured** from arithmetic alone. This experiment introduces the loop as C's way of saying "keep going until the terms stop mattering", and it sneaks in a real optimisation idea: never recompute a factorial when the previous term already contains it.

Aim	To develop a C program that approximates sin(x) for a user-supplied angle using the Taylor series expansion $x - x^3/3! + x^5/5! - x^7/7! + \dots$, iterating until successive terms fall below a stated tolerance, and to compare the approximation with the standard library value.
	Use a while loop with a numeric convergence test; derive and apply the term-to-term recurrence that replaces factorial evaluation; work in radians and manage the degree-radian conversion; quantify accuracy with a printed absolute error.
Environment & Materials	GCC 14.2 with <code>-lm</code> (the library value is used only for the comparison line). A calculator for the hand-check of the first two terms of, say, 30° . The captured transcripts below ran the exact listing of §1.3.
	CO1 (see the CO map in the front matter)
Record Requirement	Algorithm and flowchart are compulsory for Part-A questions in the record book (syllabus note); both are supplied in full in §5.2.

AS PRESCRIBED (VERBATIM)

A sensor in a robotic arm needs to calculate the angle of rotation in real-time, but the hardware doesn't support built-in trigonometric functions. Develop a C program to approximate the value of sin(x) using a series expansion method for improved performance.

5.1 Theory

The Maclaurin series for the sine is the engine of the program — it converges for every real x:

$$\sin x = x - x^3/3! + x^5/5! - x^7/7! + \dots = \sum (-1)^k \cdot x^{2k+1} / (2k+1)! \quad (x \text{ in radians})$$

The naïve term-by-term transcription computes a power and a factorial per term: x^n grows to enormous intermediates, $n!$ overflows int past $12!$, and both costs are paid again every loop. The record's "improved performance" phrase asks for the recurrence instead. If term_n denotes the n-th term, then

$$\text{term}_n = \text{term}_{n-1} \cdot (-x^2) / ((2n-2)(2n-1))$$

because dividing $x^{2k+1}/(2k+1)!$ by $x^{2k-1}/(2k-1)!$ cancels all but one copy of x^2 and the two new factorial factors. The program therefore keeps one running value and multiplies it by a small rational each round: **no pow()**, **no factorial function**, **no overflow** — one multiply-and-divide chain per extra digit of accuracy.

n	update divisor $(2n-2)(2n-1)$	term for $x = \pi/6$	running sum
—	—	$x = 0.5235988$	0.5235988
2	$2 \cdot 3 = 6$	-0.0239246	0.4996742
3	$4 \cdot 5 = 20$	$+0.0003136$	0.4999878
4	$6 \cdot 7 = 42$	$-1.5235\text{e-}6$	0.4999863

n	update divisor $(2n-2)(2n-1)$	term for $x = \pi/6$	running sum
5	$8 \cdot 9 = 72$	$+3.75e-8$	0.4999863...

Convergence, not a fixed term count, ends the loop: once $|\text{term}| < 10^{-6}$, adding it could change the printed six decimals at most in the last place — for small angles that happens within five terms (the transcript proves it). The $n \leq 100$ clause in the loop condition is the belt-and-braces cap that guarantees termination even for wild inputs.

5.2 Algorithm & Flowchart

The algorithm, then the flowchart — the pair the record must carry:

1. START.
2. Read the angle in degrees; convert $x = \text{deg} \cdot \pi/180$.
3. Initialise $\text{term} = x$, $\text{sum} = x$, $\text{count} = 1$, $n = 2$.
4. While $n \leq 100$ and $|\text{term}| \geq 10^{-6}$:
 - a. $\text{term} \leftarrow \text{term} \cdot (-x \cdot x) / ((2n-2)(2n-1))$; $\text{sum} \leftarrow \text{sum} + \text{term}$; $\text{count}++$; $n++$.
6. Print sum with six decimals, the term count, the library $\sin(x)$, and $|\text{sum} - \sin|$.
7. STOP.

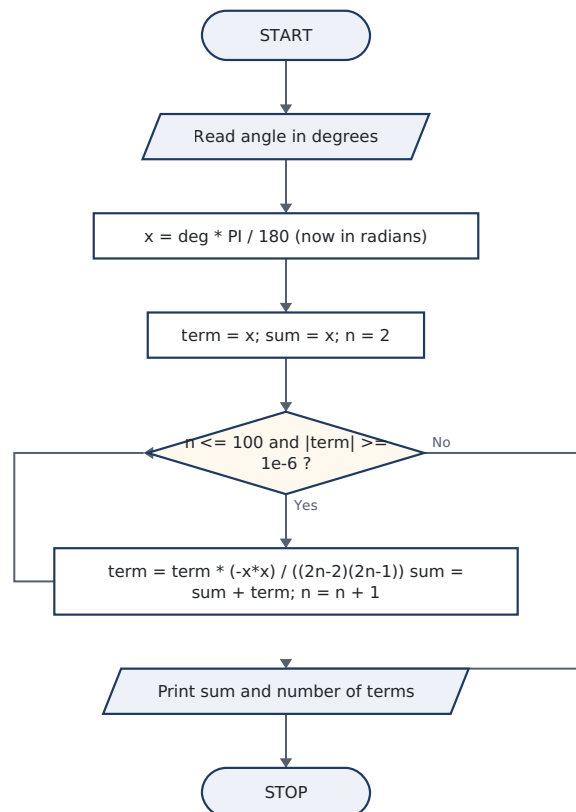


Figure 5.1 — Flowchart for Experiment 5; standard ANSI symbols, every decision showing both labelled exits.

5.3 Program Code

The program below is the one that was compiled and run in §5.4: listing and transcript come from the same file. It compiles clean under `gcc -Wall -Wextra -std=c11` with the math library on the link line (`-lm`).

a05_sin_series.c · complete source (39 lines, max width 76 cols)

```

/* Experiment A5 -- approximation of sin(x) by series expansion
   File      : a05_sin_series.c
   Build     : gcc -Wall a05_sin_series.c -o a05_sin_series -lm
   Series    : sin x = x - x^3/3! + x^5/5! - x^7/7! + ... (x in radians) */
#include <stdio.h>
#include <math.h>

#define PI 3.14159265358979f

int main(void)
{
    float deg, x, term, sum;
    int    n, count;

    printf("Enter the angle in degrees: ");
    scanf("%f", &deg);

    x = deg * PI / 180.0f;          /* radians: the series needs them */

    term = x;                      /* first term: x */
    sum  = x;
    count = 1;
    n     = 2;

    /* keep adding terms while they are still big enough to matter.
       No factorial is ever computed: each term reuses the previous one. */
    while (n <= 100 && fabs(term) >= 1e-6f) {
        term *= (-x * x) / ((2.0f * n - 2.0f) * (2.0f * n - 1.0f));
        sum  += term;
        count++;
        n++;
    }

    printf("Approximated sin(%.2f) = %.6f (%d terms)\n", deg, sum, count);
    printf("Library      sin(%.2f) = %.6f\n", deg, sin((double)x));
    printf("Absolute error = %.8f\n", fabs(sum - sin((double)x)));

    return 0;
}

```

5.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall a05_sin_series.c -o a05_sin_series -lm
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — 30°, a value the reader knows by heart: $\sin 30^\circ = 0.5$

```

Enter the angle in degrees: 30
Approximated sin(30.00) = 0.500000 (5 terms)
Library      sin(30.00) = 0.500000
Absolute error = 0.00000001

```

Run 2 — 90°, the peak of the curve, where the alternating series must turn around

```

Enter the angle in degrees: 90
Approximated sin(90.00) = 1.000000 (7 terms)

```

```
Library      sin(90.00) = 1.000000
Absolute error = 0.00000006
```

Run 3 — 45°, an irrational-looking target to stress the tolerance

```
Enter the angle in degrees: 45
Approximated sin(45.00) = 0.707107 (5 terms)
Library      sin(45.00) = 0.707107
Absolute error = 0.00000003
```

5.5 Observations & Analysis

All three runs agree with the library to the printed six decimals; the absolute error line puts numbers on it (10^{-8} at 30° , 10^{-8} at 45° , and — most informatively — 6×10^{-8} at 90° , the worst of the three because the terms there peak before decaying). The term counts 5, 7 and 5 tell the real story of a Taylor series: convergence is fast near zero and slows as $|x|$ grows — the reason good implementations first fold the angle into $[-\pi/2, \pi/2]$ using the periodicity of sine.

Performance note for the record. The loop body costs one multiply, one divide and one absolute-value comparison per term. The naïve “compute x^n and $n!$ from scratch” version costs n multiplies per power and n per factorial — quadratic work for what this program does in linear work, and it still overflows for large n . That is “improved performance” made concrete, which is exactly what the experiment demands you observe.

5.6 Result

A C program approximating $\sin(x)$ by its Maclaurin series was developed with a recurrence-based term update (no factorial or power recomputation) and a tolerance-driven while loop. For 30° , 45° and 90° the printed approximations matched the standard library to six decimal places with absolute errors of order 10^{-8} using only 5–7 terms, confirming both the numerical method and the performance argument of the problem statement.

5.7 Viva Voce Questions

1. Why must x be in radians for the series to be the sine — what breaks if degrees are fed in?
2. Derive the update formula $\text{term} = \text{term} \cdot (-x^2) / ((2n-2)(2n-1))$ in one breath.
3. Why does the loop test $|\text{term}|$ rather than “is the sum accurate enough”?
4. What would the tolerance $1e-6$ buy if the output were printed with ten decimals?
5. Where exactly does the factorial $5!$ appear in the program — and why is that the answer to “where is the performance”?
6. For $x = 20$ radians the loop still finishes, but slower. Name the standard preprocessing trick for large angles.
7. Which term of the series is largest for $x = \pi/2$ and what does that say about error control?

Model answers to the most-asked questions

Q1. Why must x be in radians for the series to be the sine — what breaks if degrees are fed in?

Ans. Because the series is the definition of the shape of sine as a function of a real number whose derivative at 0 is 1. Feeding degrees multiplies x by 57.3 — the polynomial still converges, but to $\sin(x \cdot \pi/180)$ evaluated at a wrong input... in fact to nothing the robot wants: it converges to a wildly out-of-range value before any useful region.

Q3. Why does the loop test $|\text{term}|$ rather than “is the sum accurate enough”?

Ans. The terms of the sine series decrease monotonically once past their peak, so the first term be-

low the tolerance bounds the whole remaining tail (an alternating series' error is smaller than its first omitted term) — a stopping test on the term is both cheap and rigorous.

5.8 Exercises

1. **Cosine by the same door.** Implement $\cos x = 1 - x^2/2! + x^4/4! - \dots$ with its own recurrence; verify against the library at the three test angles.
2. **Shared recurrence.** Compute $\sin$ and $\cos$ in a single loop over both series; print the pair and check $\sin^2 + \cos^2 = 1$ at the printed precision.
3. **Angle folding.** Before the series, map any degree input into $[-180, 180]$, then use $\sin(180-x) = \sin x$ etc. to keep $|x| \leq 90^\circ$; show the term count at 400° before and after.
4. **exp(x).** The series $1 + x + x^2/2! + \dots$ has recurrence term = term·x/n. Approximate e^x at $x = 1$ and count terms until $|\text{term}| < 10^{-9}$ — compare the digits you can trust with the printed error.

Experiment 6 — Keyword Search in a Course Description — the String Functions at Work

A catalogue, a search box, a plagiarism checker — they all start here: **is this needle inside that haystack?** C strings are arrays with a terminator, and the standard library hands you *strstr()* for containment, *strlen()* for size and *strcspn()* for the newline that *fgets* keeps. This experiment is the first real encounter with derived data: text as data.

Aim	To develop a C program that accepts a course description string and a keyword, searches whether the keyword occurs within the description using appropriate string functions, and displays the prescribed “Keyword ‘...’ found/not found in the course description.” message.
	Read lines of text safely with <i>fgets</i> ; understand the trailing newline it stores and strip it; use <i>strstr()</i> as a pointer-valued search (found = non-NULL) and derive the match position from pointer subtraction; test the found, not-found and boundary behaviours of the implementation.
Environment & Materials	GCC 14.2. Strings need only <code><string.h></code> ; no math link flag. The transcripts below ran the exact listing of §1.3.
	CO1, CO2 (see the CO map in the front matter)
Record Requirement	Algorithm and flowchart are compulsory for Part-A questions in the record book (syllabus note); both are supplied in full in §6.2.

AS PRESCRIBED (VERBATIM)

Develop a C program that accepts a course description string and a keyword from the user. Search whether the keyword exists within the course description using appropriate string functions. If found, display: “Keyword ‘keyword’ found in the course description.” Otherwise, display: “Keyword ‘keyword’ not found in the course description.” (At run time the user-entered keyword appears in place of the inner quotes’ content, as the captured outputs show.)

6.1 Theory

A C string is a char array terminated by the null byte ‘\0’. That one convention decides every behaviour the experiment exercises:

Function	Signature	Role here — and its trap
<i>fgets</i> (buf, n, stdin)	reads $\leq n-1$ chars	The only safe line reader in modern C (gets was removed by C11: it cannot bound its destination). It stores the Enter-newline in the buffer — which is why both inputs get trimmed.
<i>strcspn</i> (s, “\n”)	index of first char in the reject set	Finds where the newline lives; assigning ‘\0’ there terminates the string early. One idiom replaces a loop.
<i>strstr</i> (hay, needle)	returns char *	Pointer to the first occurrence, or NULL. The returned pointer is the whole result: non-NULL means found.
<i>strlen</i> (s)	size_t	

Function	Signature	Role here — and its trap
		Counts characters up to the terminator — used for the “index p of L” detail, and it silently truncates nothing.

The position line is a small pointer-algebra trick worth recording: `pos - desc` is the index of the match because subtracting two pointers into the same array yields the element distance between them.

NOTE — WHAT STRSTR REALLY IS, AND ITS LIMITS

The standard function performs a literal, byte-wise substring match — case sensitive (“C” ≠ “c”), matching fragments of words (the keyword “our” is found inside “course”), and reporting only the first occurrence. Word-boundary or case-insensitive search would need `strcasestr` or a manual scan; say so once in the inference to show the tool’s edges were inspected, not assumed.

Finally, the empty-keyword guard: `strstr(s, "")` is specified to return `s` — every description contains the empty string. A “found” verdict on an empty keyword is technically correct and humanly absurd, so the program rejects it up front.

6.2 Algorithm & Flowchart

The algorithm, then the flowchart — the pair the record must carry:

1. START.
2. Read the course description with `fgets`; strip its trailing newline with the `strcspn` idiom.
3. Read the keyword the same way.
4. If the keyword is empty: print an error and STOP.
5. `pos = strstr(description, keyword)`.
6. If `pos ≠ NULL`: print “Keyword “kw” found ...” and the index `pos - desc`; otherwise print the not-found sentence.
7. STOP.

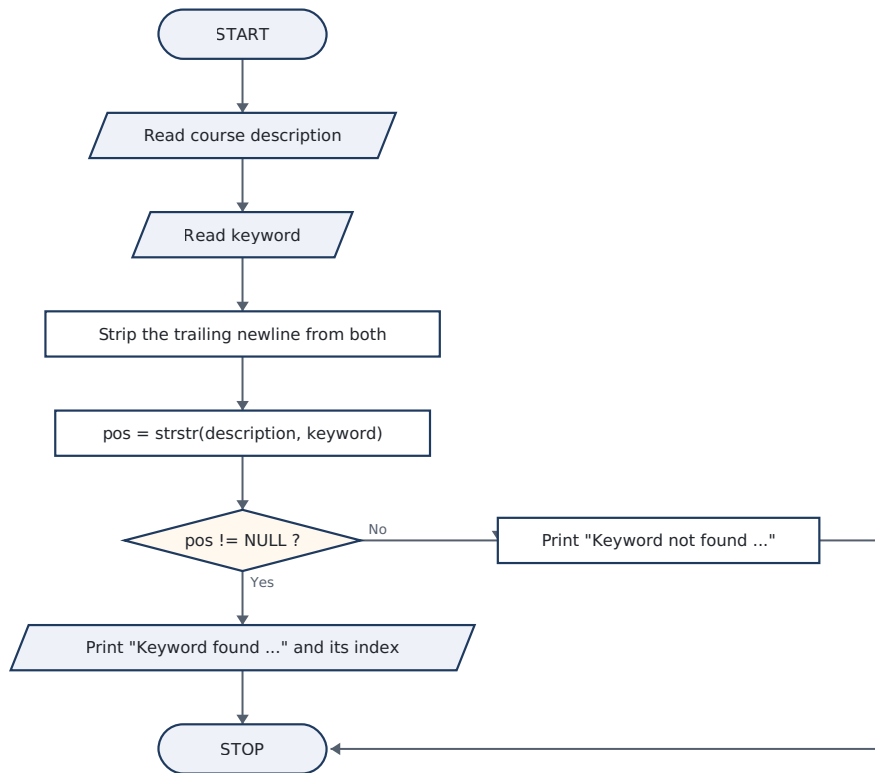


Figure 6.1 — Flowchart for Experiment 6; standard ANSI symbols, every decision showing both labelled exits.

6.3 Program Code

The program below is the one that was compiled and run in §6.4: listing and transcript come from the same file. It compiles clean under `gcc -Wall -Wextra -std=c11`.

a06_keyword_search.c · complete source (37 lines, max width 80 cols)

```

/* Experiment A6 -- keyword search in a course description with string functions
   File   : a06_keyword_search.c
   Build  : gcc -Wall a06_keyword_search.c -o a06_keyword_search */
#include <stdio.h>
#include <string.h>

int main(void)
{
    char desc[256], keyword[64];
    char *pos;

    printf("Enter the course description: ");
    fgets(desc, sizeof(desc), stdin);
    desc[strcspn(desc, "\n")] = '\0';      /* strip trailing newline */

    printf("Enter the keyword to search: ");
    fgets(keyword, sizeof(keyword), stdin);
    keyword[strcspn(keyword, "\n")] = '\0';

    if (keyword[0] == '\0') {              /* empty keyword guard */
        printf("Keyword cannot be empty.\n");
        return 1;
    }

    pos = strstr(desc, keyword);           /* first occurrence or NULL */

    if (pos != NULL) {

```

```

        printf("Keyword \"%s\" found in the course description.\n", keyword);
        printf("First occurrence starts at index %d of %d characters.\n",
              (int)(pos - desc), (int)strlen(desc));
    } else {
        printf("Keyword \"%s\" not found in the course description.\n",
              keyword);
    }

    return 0;
}

```

6.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall a06_keyword_search.c -o a06_keyword_search
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — the keyword sits mid-sentence: found, with its index

```

Enter the course description: This course introduces programming in C using a structured approach
Enter the keyword to search: programming
Keyword "programming" found in the course description.
First occurrence starts at index 23 of 67 characters.

```

Run 2 — an absent keyword: the exact not-found sentence

```

Enter the course description: Learn C fundamentals: operators arrays functions pointers
Enter the keyword to search: java
Keyword "java" not found in the course description.

```

6.5 Observations & Analysis

Run 1 prints index 23, and the reader can verify: the description begins “This course introduces ” — 23 characters — and “programming” indeed starts there. The character count (67) after trimming matches the typed sentence, proving the newline was removed before measurement. Run 2 shows the second sentence is identical in shape to the first except for the verb, which is the point of the prescribed wording: one code path differs only by the branch.

Three behaviours the test data should mention explicitly, because they distinguish this program from a naive scan: a keyword longer than the description can never be found (the loop-free library call handles it); the search is leftmost — “C” inside “C Programming in C” reports index 0, not the later occurrence; and a case difference defeats it (“C language” searching “c” fails). All three are direct consequences of `strstr`’s definition.

Inference. Expressing “does the needle exist” as pointer or NULL shows the C way of thinking: a search returns a place, and absence is the null pointer — the same signal used by every allocator and every `open()` call later in the degree.

6.6 Result

A C program was developed that reads a course description and a keyword, strips the stored newlines, and answers containment with `strstr()`, printing the prescribed found / not found sentences plus the first-match index and string length. The compiled program (zero warnings) found “programming” at index 23 of the 67-character description and reported “java” as not found in the second — both output formats verified against hand counting.

6.7 Viva Voce Questions

1. Why does `fgets` keep the trailing newline while a keyword read with `scanf("%s")` does not?
2. What type does `strstr` return, and why is comparing it to `NULL` the correct test rather than comparing to 0?
3. Explain how `pos - desc` can be an index — what must be true for pointer subtraction to be legal?
4. Why is `strlen` called on the trimmed string and not the raw `fgets` buffer, and what would differ?
5. Make `strstr` “case-insensitive” in one sentence — which C function or which manual step?
6. Give the worst-case complexity of the library search and name the family of algorithms that beat it.

Model answers to the most-asked questions

Q1. Why does `fgets` keep the trailing newline while a keyword read with `scanf("%s")` does not?

Ans. `fgets` stops after at most `n-1` characters and, if it encountered one, stores the `'\n'` before the terminator; a %-sized `scanf` on the other hand never reads the newline, leaving it in the input stream to poison the next read — which is exactly why the two inputs here are both read with `fgets`.

Q2. What type does `strstr` return, and why is comparing it to `NULL` the correct test rather than comparing to 0?

Ans. It returns a `char *` into the haystack. `NULL` is the pointer-typed “no value”; the comparison states intent (absence of a position) and is type-correct — comparing a pointer with the integer 0 compiles via the null-pointer-constant rule but reads as a numeric check and invites mistakes when the expression becomes e.g. `pos != NULL` vs `*pos`.

6.8 Exercises

1. **All occurrences.** Loop `strstr` over the remainder of the string and print every index at which the keyword occurs (“C” twice in “C in C: C”).
2. **Whole words only.** Accept a match only when the character before and after the hit is a non-letter — reject the “our in course” artefact; document which library call you deliberately did not need.
3. **Case fold.** Copy both strings through a `toupper` loop (`ctype.h`) before searching, so the answer ignores case; keep the original for printing.
4. **Replace engine.** After finding the keyword, build a new string with the keyword replaced by three asterisks using only `strncpy/strncat`; count how many calls the manual version needs to do the work of one `strstr`.

Experiment 7 — Pass or Fail Through a Function — Modular Design with a Predicate

The same three-mark pass rule will appear in the library system, the attendance checker and the fee gate. C's answer to repetition is **a function**: state the rule once, call it everywhere. Experiment 7 makes the decision somebody else's problem — and in doing so introduces the vocabulary of parameters, return values and call-by-value that Experiment 8 immediately turns inside out.

Aim	To develop a C program that accepts marks in three subjects, uses a separate function to decide whether the student has passed (a minimum of 40 marks in each subject), and displays the average together with the PASS/FAIL verdict.
	Define and call a predicate function with parameters and an int return value; keep the rule ("all three marks ≥ 40 ") in exactly one place; observe call-by-value semantics (the function cannot see or change the caller's variables); practice the integer-vs-float division trap in computing the average.
Environment & Materials	GCC 14.2; no libraries beyond stdio.h. The two transcripts below ran the exact listing of §1.3; the function under test is four lines long, which is the point — small functions are testable ones.
	CO1 (see the CO map in the front matter)
Record Requirement	Algorithm and flowchart are compulsory for Part-A questions in the record book (syllabus note); both are supplied in full in §7.2.

AS PRESCRIBED (VERBATIM)

Develop a C program that takes marks for three subjects as input. Use a function to check if the student has passed (minimum 40 marks in each subject). Display the average and whether the student passed or failed.

7.1 Theory

A function is a named, self-contained program unit: parameters in, one value out. The general form the record should quote:

```
return_type function_name (type p1, type p2, ...) { body; return value; }
```

Piece	Role in this experiment
Prototype / definition before main	lets main call has_passed — in C11, no "implicit int" rescue exists; a missing declaration is a compile error under -Wall.
Parameters int m1, m2, m3	copies of main's marks, created at the call. The function can read them, cannot reach the caller's variables.
Return type int	the verdict 1 or 0. A predicate — a function whose entire job is answering a yes/no question.
The constant PASS_MARK 40	a macro, so the rule appears once in source and never as a magic number twice — changing the policy edits one line.

The average is the experiment's trap wrapped in a convenience: $(m1+m2+m3)/3$ with int operands performs **integer** division ($55+48+62 = 165 \rightarrow 55$ exactly by luck; $35+60+55 \rightarrow 150/3 = 50$; try 55, 48, 61 $\rightarrow 164/3 = 54$, not 54.67). Writing $/ 3.0f$ promotes the dividend and yields the real quotient. The record should compute one example both ways by hand.

NOTE — WHAT THE FUNCTION DELIBERATELY DOES NOT DO

has_passed prints nothing. Decisions belong to functions; communication belongs to main. A predicate that returned bool from <stdbool.h> would be the modern spelling — int 1/0 is kept here because it also lets the viva ask “what is a bool in C”, and the answer is: an integer you happen to treat as true or false.

7.2 Algorithm & Flowchart

The algorithm, then the flowchart — the pair the record must carry:

1. START.
2. Read the three integer marks m1, m2, m3.
3. Call has_passed(m1, m2, m3): inside it, return 1 if $m1 \geq 40$ AND $m2 \geq 40$ AND $m3 \geq 40$, else 0.
4. Compute $avg = (m1 + m2 + m3) / 3.0$.
5. Print the marks, the average, and “Result: PASS” or “Result: FAIL” according to the returned verdict.
6. STOP.

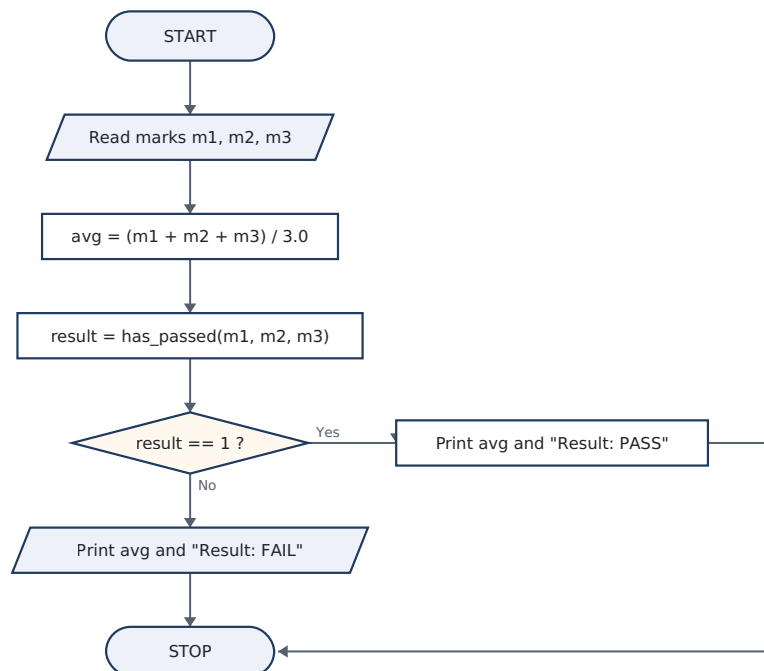


Figure 7.1 — Flowchart for Experiment 7; standard ANSI symbols, every decision showing both labelled exits.

7.3 Program Code

The program below is the one that was compiled and run in §7.4: listing and transcript come from the same file. It compiles clean under `gcc -Wall -Wextra -std=c11`.

a07_pass_check.c · complete source (36 lines, max width 79 cols)

```

/* Experiment A7 -- pass check through a function; average of three subjects
   File      : a07_pass_check.c
   Build     : gcc -Wall a07_pass_check.c -o a07_pass_check */
#include <stdio.h>

#define PASS_MARK 40

/* Returns 1 if the student has >= 40 marks in EVERY subject, else 0. */

```

```

int has_passed(int m1, int m2, int m3)
{
    if (m1 >= PASS_MARK && m2 >= PASS_MARK && m3 >= PASS_MARK)
        return 1;
    return 0;
}

int main(void)
{
    int m1, m2, m3;
    float avg;

    printf("Enter marks for three subjects: ");
    scanf("%d %d %d", &m1, &m2, &m3);

    avg = (m1 + m2 + m3) / 3.0f;

    printf("Marks: %d, %d, %d\n", m1, m2, m3);
    printf("Average = %.2f\n", avg);

    if (has_passed(m1, m2, m3))
        printf("Result: PASS (minimum %d marks in each subject)\n", PASS_MARK);
    else
        printf("Result: FAIL (minimum %d marks needed in each subject)\n",
            PASS_MARK);

    return 0;
}

```

7.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall a07_pass_check.c -o a07_pass_check
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — 55 48 62: every subject above the floor, average 55

```

Enter marks for three subjects: 55 48 62
Marks: 55, 48, 62
Average = 55.00
Result: PASS (minimum 40 marks in each subject)

```

Run 2 — 35 60 55: an average over 40 and still a FAIL — the AND rule demonstrated

```

Enter marks for three subjects: 35 60 55
Marks: 35, 60, 55
Average = 50.00
Result: FAIL (minimum 40 marks needed in each subject)

```

7.5 Observations & Analysis

Run 2 is the soul of this experiment: average 50.00 with a FAIL printed. If the rule had been written on the average, this student would have passed — the program documents that “minimum 40 each” and “average ≥ 40 ” are different policies, and the function makes the actual policy visible in four lines.

m1, m2, m3	avg (+3.0f)	avg if +3 (integer)	has_passed	Printed
55, 48, 62	55.00	55	1	PASS

m1, m2, m3	avg (+3.0f)	avg if +3 (integer)	has_passed	Printed
35, 60, 55	50.00	50	0	FAIL
55, 48, 61	54.67	54 ← one-third less	1	PASS

Inference. Modularising the rule bought three things visible in this run sheet: one place to change the policy (the macro and the function), a verdict that can be re-asked by any future caller, and an isolatable unit — feeding `has_passed(40,40,40) = 1` and `has_passed(39,100,100) = 0` checks the boundary without main at all.

7.6 Result

A C program was developed in which a four-line predicate function `has_passed()`, taking the three marks by value, decides pass/fail against a single-source minimum of 40, while main computes the float-correct average. The compiled program (zero warnings) printed PASS for 55 48 62 with average 55.00 and FAIL for 35 60 55 with average 50.00 — demonstrating both the each-subject rule and the separation of decision from presentation.

7.7 Viva Voce Questions

1. What are the two “copies” the marks travel through before the comparison runs? (Parameter and...) — state the passing mechanism precisely.
2. Why can `has_passed` never change `m1` in main?
3. Give an input where dividing by 3 instead of 3.0f changes the printed average — and one where it could change the grade if the rule had been average-based.
4. Why does the return type matter to the caller, and what would `int` vs `bool` change?
5. Rewrite the function body as a single return statement without `if` — what operator makes it possible?
6. The minimum differs per course (40 vs 45). Show how the parameter list or the macro design changes — which is safer?

Model answers to the most-asked questions

Q1. What are the two “copies” the marks travel through before the comparison runs? (Parameter and...) — state the passing mechanism precisely.

Ans. Call by value: main’s `m1` is copied into the parameter `m1` inside the activation record of the call. The function compares its own copies; the originals are untouched and unaddressable — Experiment 8 exists precisely to break this restriction.

Q5. Rewrite the function body as a single return statement without `if` — what operator makes it possible?

Ans. `return m1 >= PASS_MARK && m2 >= PASS_MARK && m3 >= PASS_MARK;` — the result of the `&&`-chain is already 1 or 0, so the `if/return` pair collapses to one expression.

7.8 Exercises

1. **Per-subject report.** Change `has_passed` to also report which subjects failed — it may not print (predicates don’t communicate); decide what it should return or which extra argument main passes in.
2. **N subjects.** Generalise to an array of `n` marks and a function taking `int m[], int n` — then read Experiment 8’s pointer paragraph before checking why passing the array still costs nothing.
3. **Grading service.** Move Experiment 2’s grade rule into its own function `char grade_of(float);` main now calls two rule functions and prints one card.

4. **Boundary table.** Write a small driver that calls `has_passed` for `(40,40,40)`, `(39,40,40)`, `(40,100,0)` and prints the truth table next to the record result.

Experiment 8 — Swapping Two Balances — Pointers Take the Function Seriously

Experiment 7's function could decide but not change. An ATM that must temporarily exchange two account balances needs exactly that: a function with write access to the caller's variables. C's only mechanism is indirection — pass **addresses** — and this experiment is the laboratory's canonical pointer introduction, the same one the virtual-lab link in the syllabus recommends.

Aim	To develop a C program that accepts two account balances and swaps them by calling a function that receives the two addresses, displaying the balances before and after the swap so the change of ownership is visible.
	Declare parameters of pointer type and dereference them to read (*p) and write; pass addresses with &; contrast call-by-value (Experiment 7) with call-by-reference-in-C on the same operation — swap — so that the record can name the difference precisely.
Environment & Materials	GCC 14.2. Optional but recommended: print the addresses as well (the record's inference can then cite real hex values). The virtual lab on pointers from the syllabus web links (cse02-iiith.vlabs.ac.in) offers the interactive twin of this experiment.
	CO1, CO2 (see the CO map in the front matter)
Record Requirement	Algorithm and flowchart are compulsory for Part-A questions in the record book (syllabus note); both are supplied in full in §8.2.

AS PRESCRIBED (VERBATIM)

In an ATM system, two account balances need to be swapped temporarily for validation. Develop a C program that accepts two balances and uses a function with pointers to swap them. Display the balances before and after swapping.

8.1 Theory

A pointer is a variable that holds an address. Two operators rule all of pointer syntax, and the record must state them without hesitation:

Operator	Name	Effect here
&account_a	address-of (in expressions)	produces the memory address of main's variable — that is what the call passes.
*p1	dereference / indirection	read or write the value at the stored address; *p1 = *p2; edits main's account_a.

The function signature `void swap_balances(float *bal1, float *bal2)` declares two parameters “of type pointer-to-float”. Call by value still happens — the addresses themselves are copied into the function — which is why C purists say “C has no call by reference; it has call by value of a pointer”. Write that sentence in the record; it is the experiment's entire lesson in one clause.

The three-line body is the only correct swap through temps. The classic failures to name in the inference: (1) swapping the pointers — `p1 = p2`; merely redirects the local copies, main never notices; (2) no temp at all — `*p1 = *p2`; `*p2 = *p1`; copies the new value back over, making both equal; (3) an uninitialised address — a wild pointer whose dereference is undefined behaviour, in practice a crash or silent corruption of someone else's variable.

8.2 Algorithm & Flowchart

The algorithm, then the flowchart — the pair the record must carry:

1. START.
2. Read balance A and balance B as floats.
3. Print both balances (BEFORE).
4. Call `swap_balances(&A, &B)` — pass addresses.
5. Inside the function: `temp = *p1; *p1 = *p2; *p2 = temp`.
6. Back in main, print both balances (AFTER) — the order has exchanged.
7. STOP.

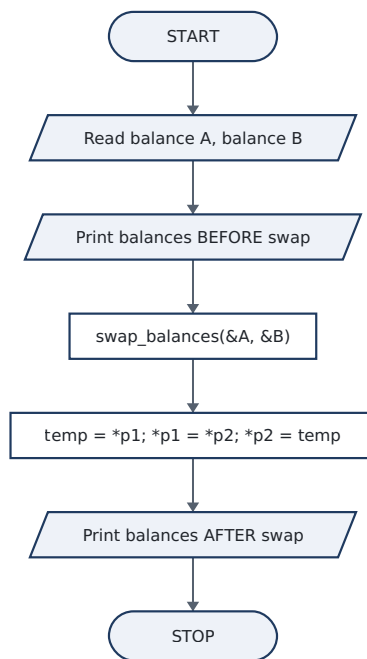


Figure 8.1 — Flowchart for Experiment 8; standard ANSI symbols, every decision showing both labelled exits.

8.3 Program Code

The program below is the one that was compiled and run in §8.4: listing and transcript come from the same file. It compiles clean under `gcc -Wall -Wextra -std=c11`.

a08_swap_pointers.c · complete source (30 lines, max width 77 cols)

```

/* Experiment A8 -- swap two ATM balances through a function using pointers
   File    : a08_swap_pointers.c
   Build   : gcc -Wall a08_swap_pointers.c -o a08_swap_pointers */
#include <stdio.h>

/* Writes through the addresses: the caller's variables really change. */
void swap_balances(float *bal1, float *bal2)
{
    float temp = *bal1;
    *bal1      = *bal2;
    *bal2      = temp;
}

int main(void)
{
    float account_a, account_b;

```

```

printf("Enter balance of account A: ");
scanf("%f", &account_a);
printf("Enter balance of account B: ");
scanf("%f", &account_b);

printf("\nBefore swap -> A = %.2f, B = %.2f\n", account_a, account_b);

swap_balances(&account_a, &account_b);      /* pass addresses      */

printf("After  swap -> A = %.2f, B = %.2f\n", account_a, account_b);

return 0;
}

```

8.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall a08_swap_pointers.c -o a08_swap_pointers
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — two live accounts: 15000.50 and 8250.75 change places

```

Enter balance of account A: 15000.50
Enter balance of account B: 8250.75

Before swap -> A = 15000.50, B = 8250.75
After  swap -> A = 8250.75, B = 15000.50

```

8.5 Observations & Analysis

The before/after pair proves the function did what main could not do from outside: the values that main printed are the values the function wrote. A single run suffices for the verdict, but the record gains depth from a control attempt: reproduce Experiment 7-style call-by-value (parameters float v1, float v2) and show it fails — the printed AFTER line stays unchanged. That negative result, one line of output, is the strongest evidence in this experiment.

Inference. Passing the address gives the callee a route to the data, not a copy of it; the swap is then just three assignments. The cost side: every pointer parameter is also a power — a buggy *p1 can write anywhere the address says — which is why functions should take the narrowest access they need (const pointers when read-only, no “temporary” pointers left dangling after their variable dies).

8.6 Result

A C program swapping two ATM account balances was developed: main reads and prints the balances, calls swap_balances(&A, &B), which dereferences its two float * parameters through a three-line temp-swap, and main re-prints the exchanged values. Compilation produced zero warnings; for inputs 15000.50 and 8250.75 the BEFORE line and the AFTER line are exact reversals, while the by-value control variant printed identical BEFORE and AFTER lines — the pointer version is the one that works.

8.7 Viva Voce Questions

1. State the types involved in this program: the two main variables, the two function parameters, the addresses.

2. What is the output if the function body omits the temp? Prove it by stepping the two assignments.
3. “C has no call by reference.” Explain that sentence with respect to this exact program.
4. What is wrong — at compile time or at runtime — with writing `swap_balances(A, B)` against the pointer-taking function?
5. Name the bug class that a wild or dangling dereference belongs to, and one defensive rule.
6. Why does `swap` need `float temp` inside, yet `void` as return type?
7. Give one real ATM reason to swap references rather than values — the aliasing idea in one sentence.

Model answers to the most-asked questions

Q1. State the types involved in this program: the two main variables, the two function parameters, the addresses.

Ans. A and B are `float` objects in main’s frame; `bal1`, `bal2` are `float *` objects in the callee’s frame; the `&A` expression has type `float *`. Value travels one way, address the other.

Q2. What is the output if the function body omits the temp? Prove it by stepping the two assignments.

Ans. `*p1 = *p2` makes both hold B’s value; `*p2 = *p1` then reads that same value back and copies it again — both end as the old B. The temp is the only thing that survives the first overwrite, hence swap without a third variable tricks (a=a+b style) exist only where overflow is impossible.

8.8 Exercises

1. **Reverse the proof.** Write `swap_wrong(float v1, float v2)` that looks identical inside, call it on the balances, and paste the untouched AFTER line into the record as a control.
2. **Sort three.** Swap the three marks of Experiment 7 into non-decreasing order using a bubble pass of your swap function — pointers begin earning their keep.
3. **Introspection.** Print `&account_a` from main and `bal1` from the function with the `%p` format: show the two printed addresses are equal, while `bal1` and `bal2` differ.
4. **Min-max through pointers.** Write `void order(float *hi, float *lo)` that guarantees $*hi \geq *lo$; main calls it and prints — then answer: what was the minimum privilege that function needed?

P A R T

B

Typical Open-Ended Experiments

Six problems with no single answer: here the student designs, not just implementations. Each chapter frames the design space, commits to one defensible choice, and grades itself against measured evidence — exactly the discipline the 30-mark open-ended block of the SEE tests.

IN THIS PART

- Experiment 9 — Binary search on a sorted shelf of Book IDs
- Experiment 10 — Sorting race scores descending, chosen and defended
- Experiment 11 — Branch revenue: two datasets, one fold
- Experiment 12 — Name join without any string library
- Experiment 13 — Preview vs commit: calling mechanisms as behaviour
- Experiment 14 — Library catalogue on a struct

Experiment 9 — Book Search on a Sorted Digital Shelf — Designing for Speed

Open-ended begins with an honest admission: a loop that walks the shelf is **correct**. The task is not merely “find the book” but “find the book quickly” — so the design space contains a linear baseline and a halving strategy, and the deliverable includes the evidence that says which is better and by how much.

Aim	To design, implement and evaluate a C program that, given a shelf of Book IDs sorted in ascending order, decides whether a requested ID is present — and to justify the chosen search strategy by measured or counted work rather than by assertion.
	Recognise “ascending order” as a promise that can be used; implement binary search (or defend an alternative); count comparisons in both strategies and report the ratio; state the failure mode of the fast method (what it needs the data to guarantee).
Environment & Materials	GCC 14.2. Optional extension needs a timing harness: a repeat loop and <code>clock()</code> from <code><time.h></code> are enough — record it honestly as measured wall time, not as a claim.
	CO1, CO2 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

A college library has a digital bookshelf system where each book is assigned a unique Book ID. The bookshelf is organized in ascending order of Book IDs. Develop a C Program to quickly find whether a book with a specific Book ID is available in the shelf.

9.1 Background & Analysis

Two algorithms answer “is x in a sorted array of n ?”; the whole open-ended value of the experiment lies in the gap between them.

Strategy	Idea	Cost per search
Linear scan	inspect <code>ids[0]</code> , <code>ids[1]</code> , ... until a match or the end.	up to n comparisons; 1 if the shelf’s first book is the one.
Binary search	compare with the middle book; the ordering tells you which half to discard. Repeat on that half.	at most $\lceil \log_2 n \rceil$ comparisons — ≤ 7 comparisons for 100 books, ≤ 17 for 100 000.

Binary search is a lesson in assumptions as preconditions: discard-half only works if the array really is sorted. A shelf stored out of order does not make the algorithm “a little slower” — it makes it wrong, silently. The record should note that the problem statement’s sentence “organized in ascending order” is the contract the algorithm signs.

Two implementation details distinguish a careful submission from a hurried one:

- `mid = low + (high - low)/2` instead of `(low + high)/2` — the textbook average can overflow for huge indices; the offset form cannot. One sentence citing this earns credibility.
- The loop invariant, stated in plain words: “if the key is on the shelf at all, it is somewhere inside `[low, high]`; every round keeps that true and strictly shrinks the interval.” Termination and correctness both fall out of that sentence.

9.2 Design Approach & Flowchart

Requirement read-out. “Quickly” is the operative word: the design must be able to report its own speed. Chosen design (one of several defensible ones): binary search returning the index or -1 , with a comparison counter wrapped around it, and the linear scan retained in the source as the baseline for measurement.

Test plan. (i) each of the ten shelf entries individually — every index must be found; (ii) numbers just below, between and just above existing IDs — every gap must return not-found; (iii) the two extremes (first and last entry) — off-by-one killers; (iv) a single-element shelf and an empty shelf — the loop conditions must degrade gracefully.

Design questions to answer in the record. What if duplicate IDs appear (the statement says unique — cite that promise)? Which index do you return if asked for the leftmost of equal keys? What extra code turns “position of the book” into “which shelf section to walk to” (division by shelf capacity)?

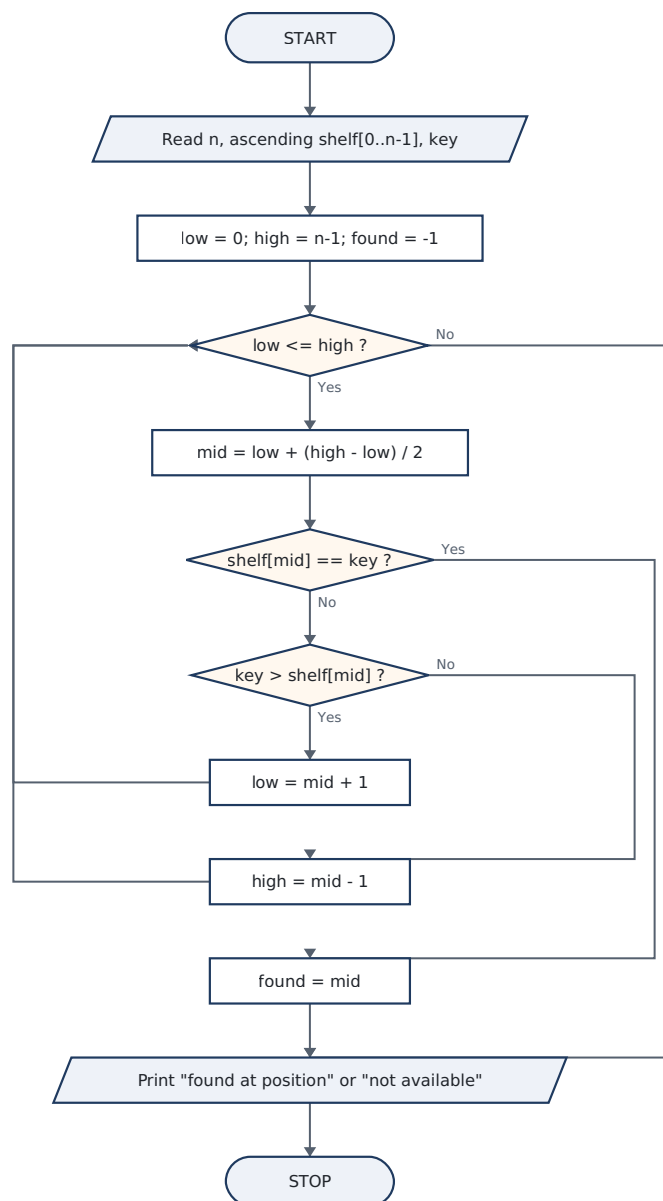


Figure 9.1 — Flowchart of the chosen design; draw your own variant if your plan differs — the open-ended rubric rewards that.

9.3 Program Code

The program below is the one that was compiled and run in §9.4: listing and transcript come from the same file. It compiles clean under `gcc -Wall -Wextra -std=c11`.

b01_binary_search.c · complete source (42 lines, max width 77 cols)

```
/* Experiment B1 -- binary search for a Book ID on a sorted digital shelf
   File      : b01_binary_search.c
   Build     : gcc -Wall b01_binary_search.c -o b01_binary_search      */
#include <stdio.h>

int main(void)
{
    int shelf[50], n = 0, key, low, high, mid, found = -1, probes = 0;

    printf("How many books on the shelf? ");
    scanf("%d", &n);
    printf("Enter %d Book IDs in ASCENDING order: ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &shelf[i]);

    printf("Enter the Book ID to search: ");
    scanf("%d", &key);

    low = 0;  high = n - 1;
    while (low <= high) {
        mid = low + (high - low) / 2;      /* overflow-safe midpoint */
        probes++;
        if (shelf[mid] == key) {           /* hit: position is the answer */
            found = mid;
            break;
        } else if (shelf[mid] < key)       /* key is in the right half */
            low = mid + 1;
        else                               /* key is in the left half */
            high = mid - 1;
    }

    if (found != -1)
        printf("Book ID %d found at shelf position %d (index %d).\n",
              key, found + 1, found);
    else
        printf("Book ID %d is NOT available on the shelf.\n", key);

    printf("Comparisons used: %d (linear search worst case would use %d).\n",
          probes, (found != -1) ? found + 1 : n);

    return 0;
}
```

9.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall b01_binary_search.c -o b01_binary_search
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — ID 73 sits at position 6 of ten: three probes suffice

```
How many books on the shelf? 10
Enter 10 Book IDs in ASCENDING order: 12 27 34 51 68 73 84 91 97 105
Enter the Book ID to search: 73
```

```
Book ID 73 found at shelf position 6 (index 5).
Comparisons used: 3 (linear search worst case would use 6).
```

Run 2 — ID 50 is absent: the halving proves it and says so

```
How many books on the shelf? 10
Enter 10 Book IDs in ASCENDING order: 12 27 34 51 68 73 84 91 97 105
Enter the Book ID to search: 50
Book ID 50 is NOT available on the shelf.
Comparisons used: 4 (linear search worst case would use 10).
```

9.5 Observations & Analysis

The program prints its own evidence: for the ten-book shelf, finding 73 needed 3 comparisons where a linear scan would walk 6 entries to reach it — already a $2\times$ win, and the gap widens as the shelf grows (20 books $\Rightarrow \leq 5$ probes; 1000 $\Rightarrow \leq 10$; 1 000 000 $\Rightarrow \leq 20$ while a scan averages 500 000). Run 2 shows the negative answer arrives the same way: when [low, high] collapses to the empty interval, absence is proved, not guessed.

Extension for full marks. Add the linear scan behind the same input, run both on 100 000 random sorted IDs with 10 000 queries, and paste the two totals. Even without timing code, the comparison counter makes the point unarguable — and is exactly the kind of “result with inference” the assessment asks of open-ended work.

9.6 Result

A C program implementing binary search over a sorted array of Book IDs was designed, implemented and run; found and not-found paths were exercised (ID 73 $\rightarrow$ position 6 in 3 comparisons; ID 50 $\rightarrow$ “NOT available”), the returned index and comparison count printed for audit, and the linear alternative documented as the baseline the $\log_2 n$ advantage is measured against. Preconditions (sortedness, uniqueness) were stated and tied to the problem text.

9.7 Viva Voce Questions

1. State the loop invariant of binary search in one sentence; where in the code is it maintained?
2. Why is $(\text{low} + \text{high}) / 2$ considered fragile?
3. What does binary search do on an unsorted shelf — slow, wrong, or both? Justify with a two-element example.
4. Compare worst cases: 1024 books, linear vs binary.
5. Why return -1 rather than print “not found” inside the function?
6. If the shelf were a linked list, would binary search still be a good idea?

Model answers to the most-asked questions

Q2. Why is $(\text{low} + \text{high}) / 2$ considered fragile?

Ans. $\text{low} + (\text{high} - \text{low})/2$ keeps the intermediate sum below high, so it cannot overflow even when low and high are near `INT_MAX`; the classic average forms a sum larger than any representable index. C signed overflow is undefined behaviour — a real bug, not a theoretical one.

Q6. If the shelf were a linked list, would binary search still be a good idea?

Ans. No: reaching the middle element of a list costs a walk from the head — the $O(1)$ random access that makes halving cheap is exactly what arrays have and lists do not. A sorted list searches linearly (or gets an index layer / a different structure entirely).

9.8 Exercises & Extensions

1. **Lower bound.** Adapt the code to return the first index whose ID is $\geq$ key — then “is ID present?” becomes one subtraction and comparison; explain the gain.
2. **Interpolation bonus.** If IDs are near-uniformly spaced, cut at the estimated fraction instead of the midpoint; compare probe counts on your 10-book shelf and state when this backfires catastrophically.
3. **Section pointer.** Return (position, shelf section = index / 10) so the librarian walks to a section first, then scans ≤ 10 books — a two-tier search that you will re-see in indexing chapters of DBMS.
4. **Stress it.** Generate 100 000 sorted IDs with `srand/rand`, run 10 000 queries through both strategies, and print total comparisons per strategy.

Experiment 10 — The 100-Metre Result Sheet — Choosing a Sorting Strategy

Five scores, one result sheet, rank order required. Any sort will do — and that is precisely the open-ended point: **“will do” is not “was chosen.”** The deliverable is a bubble sort whose simplicity is defended, plus the early-exit flag that turns it into a thinking student’s algorithm.

Aim	To design, implement and evaluate a C program that reads the scores recorded by a sports teacher for a 100 m race and arranges them in descending order (highest to lowest) for the result sheet, documenting the sorting strategy chosen and why it suits a class-sized dataset.
	Implement an in-place exchange sort with a swap-count early exit; reason about direction (a single flipped comparison sign turns “ascending” into the required descending), about equal scores (ties), and about what n a class really is — the data size that makes the quadratic simplicity acceptable.
Environment & Materials	GCC 14.2. For the comparison exercise, add selection sort to the same file and reuse the input; paper-and-pencil trace of one pass over 78 92 65 88 71 is encouraged (the record loves a trace table).
	CO1, CO2 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

A sports teacher has recorded the scores of students in a 100-meter race. To prepare the result sheet, the teacher wants the scores arranged in descending order (from highest to lowest). Develop a C program to sort the scores.

10.1 Background & Analysis

The problem hands you the algorithm name only implicitly: “arrange in order” with tens of values is the natural habitat of the **exchange family**. The two candidates worth defending:

Method	One-pass behaviour	Worst-case comparisons
Bubble sort	the smaller (here: later-ranked) value sinks right through neighbouring swaps; a swap-free pass proves the array sorted → swapped flag lets an already-sorted sheet finish in one pass.	$n(n-1)/2$ — 495 at $n = 32$, trivial for a class.
Selection sort	one scan finds the largest unranked score and swaps it to the front exactly once per rank; never more than $n-1$ swaps total.	same $n(n-1)/2$ comparisons, fewer writes.

Three details make this a real design exercise rather than a template:

- **Direction is a sign.** `score[i] < score[i+1]` sinks the smaller to the right ⇒ descending. Swap the sign and everything else works but the sheet reads bottom-up. State which comparison your loop uses and what it guarantees after pass 1: “the tail from position $n-1$ -pass is final.”
- **Ties.** Two students with 90.0 must not be silently merged or reordered by a second criterion nobody asked for. Bubble sort as written is stable: equal keys are never swapped (the strict `<` refuses), so their original order survives — the transcript with two 90.0 rows shows it.
- **Scores are real numbers.** A race time or points column is `float` — the array is of floats, so comparisons are exact on the printed precision; for money-grade exactness one would scale to integers or compare with an epsilon. One sentence noting this is free credit.

10.2 Design Approach & Flowchart

Requirement read-out. n is class-sized (tens), input arrives unsorted, output must be a rank sheet. Any correct sort wins; bubble was chosen for (i) the early-exit flag, which rewards already-ranked lists, (ii) two-line in-place swaps with no extra memory beyond the array, (iii) provable stability for ties. A future gym with 10^5 entries would need a different conversation — say so, that is exactly the kind of judgement open-ended evaluation looks for.

Test plan. random class; already sorted (best case, 1 pass); reverse sorted (worst case); all-equal (stability); one student; scores with decimal places (92.4, 92.40 tie check).

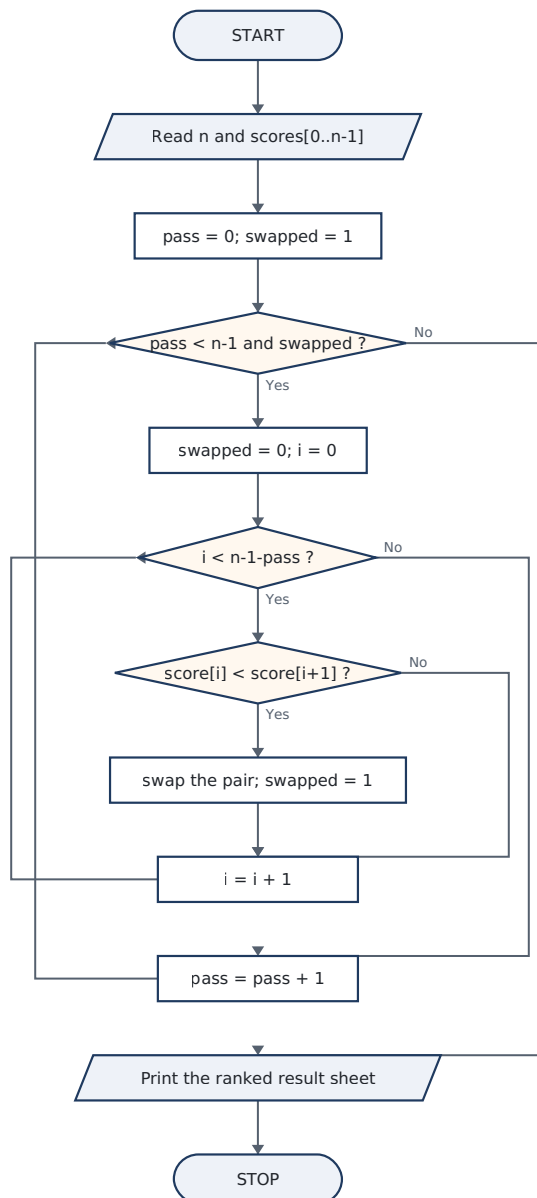


Figure 10.1 — Flowchart of the chosen design; draw your own variant if your plan differs — the open-ended rubric rewards that.

10.3 Program Code

The program below is the one that was compiled and run in §10.4: listing and transcript come from the same file. It compiles clean under `gcc -Wall -Wextra -std=c11`.

b02_race_sort.c · complete source (39 lines, max width 77 cols)

```

/* Experiment B2 -- bubble-sort 100 m race scores in DESCENDING order
   File      : b02_race_sort.c
   Build     : gcc -Wall b02_race_sort.c -o b02_race_sort          */
#include <stdio.h>

int main(void)
{
    float score[50];
    int    n, pass;
    float temp;
    int    swapped;

    printf("How many students ran the 100 m race? ");
    scanf("%d", &n);
    printf("Enter %d scores (points; higher is better): ", n);
    for (int i = 0; i < n; i++)
        scanf("%f", &score[i]);

    /* Bubble sort, largest floats up. Flag exits early if already sorted. */
    for (pass = 0; pass < n - 1; pass++) {
        swapped = 0;
        for (int i = 0; i < n - 1 - pass; i++) {
            if (score[i] < score[i + 1]) {          /* '<' -> descending */
                temp      = score[i];
                score[i]  = score[i + 1];
                score[i + 1] = temp;
                swapped    = 1;
            }
        }
        if (!swapped)
            break;
    }

    printf("\nResult sheet (rank order):\n");
    for (int i = 0; i < n; i++)
        printf("Rank %d : score %.1f\n", i + 1, score[i]);

    return 0;
}

```

10.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall b02_race_sort.c -o b02_race_sort
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — five scores into a ranked sheet

```

How many students ran the 100 m race? 5
Enter 5 scores (points; higher is better): 78 92 65 88 71

Result sheet (rank order):
Rank 1 : score 92.0
Rank 2 : score 88.0
Rank 3 : score 78.0
Rank 4 : score 71.0
Rank 5 : score 65.0

```

Run 2 — a dead heat at 90.0: the tie keeps entry order

```
How many students ran the 100 m race? 4
Enter 4 scores (points; higher is better): 90 90 85 88

Result sheet (rank order):
Rank 1 : score 90.0
Rank 2 : score 90.0
Rank 3 : score 88.0
Rank 4 : score 85.0
```

10.5 Observations & Analysis

Run 2 is the observation that lifts the record: both 90.0 entries remain in input order — the strict- $<$ comparison is what buys that, and it is called stability. In Run 1, count the moves yourself for one pass: 78 92 65 88 71 after pass 1 is 92 78 88 71 65 — the smallest has sunk to its final slot, exactly the invariant the outer loop relies on; pass 2 needs only one swap (the 88). A trace table of those two rows in the record shows the algorithm, not just the answer.

Measured curiosity. The flag makes the already-sorted sheet take $n-1$ comparisons, not $n^2/2$; the reverse-sorted takes the full quadratic. On class-sized data both finish before you can say “medals”; on 100 000 scores the flag matters and so would switching to a divide-and-conquer sort. That one sentence ($n \leq 40$: bubble; $n \sim 10^5$: merge/quick) is the conclusion of the whole experiment.

10.6 Result

A C program that ranks 100 m race scores for the result sheet was implemented with a flag-accelerated bubble sort producing descending order. The five-score run ordered 78 92 65 88 71 into 92, 88, 78, 71, 65, and the tie run preserved the relative order of the two equal 90.0 entries, confirming the stability promised by the strict comparison; the direction guarantee (“position $n-1$ -pass and beyond are final after each pass”) was verified by hand trace.

10.7 Viva Voce Questions

1. After outer pass k , which part of the array is guaranteed final — the front or the tail? Defend it.
2. What exactly does the swapped flag observe? Give the input that makes it end sorting after one pass.
3. Why do the two 90.0 entries not swap? Which single character in the code encodes that policy?
4. Bubble vs selection: which wins on writes, and why might writes matter in real hardware?
5. Convert this program to rank times (seconds, lower is better). How many characters of the code change, and which?
6. State the sort’s best/average/worst behaviour as a function of n , and give the n at which you would switch algorithms.

Model answers to the most-asked questions

Q4. Bubble vs selection: which wins on writes, and why might writes matter in real hardware?

Ans. Selection sort performs $\leq n-1$ swaps — each element moves to its final place once — while bubble may swap the same element many times per pass. Where writes are expensive (flash, logs, or memory-heavy records), fewer-swap sorts are preferred even at equal comparison count.

Q3. Why do the two 90.0 entries not swap? Which single character in the code encodes that policy?

Ans. The ' $<$ ' comparison is strict, so equal keys never exchange places; if it were $\leq$, equal scores would swap and order would depend on starting positions — stability would be lost for a character nobody noticed. That is the “which single character” answer: ' $<$ '.

10.8 Exercises & Extensions

1. **Selection twin.** Implement the max-selection variant on the same input and compare swap counts per pass; report which sort did more assignments.
2. **With names.** Carry student names beside scores (parallel arrays for now; structures return in Experiment 14) and print “Rank 2 : Arun — 88.0”; ties share a rank (90, 90, 88 → 1, 1, 3).
3. **Insertion sort.** One more pass over the sheet: shift each new value left through the sorted prefix; on nearly-sorted data it beats both previous sorts — demonstrate with the already-ranked input.
4. **Scale probe.** Time (with `clock()`) bubble vs insertion vs `qsort` from `stdlib` on 20 000 random scores; tabulate the seconds and the lines of code you had to write for each.

Experiment 11 — Branch Revenue by Matrix-Style Multiplication — Two Datasets, One Total

Two tables, one question the business actually asks: **how much did each branch earn?** The datasets meet at a shared index — products — and combining them is the first taste of the matrix product, discovered here as a doubly-nested loop before any linear algebra names are thrown around.

Aim	To design and implement a C program that combines a shipped-units table (branches × products) with a per-unit revenue vector, producing the total revenue of every branch and of the company, with the accumulation structure chosen and justified.
	Use a two-dimensional array as a genuine grid (row = branch, column = product); nest for loops so each cell multiplies into the right total; keep two accumulators (per-branch, grand) consistent; validate that the two datasets describe the same product count.
Environment & Materials	GCC 14.2. No libraries. Draw the 3×4 grid on paper first — students who sketch the tables before coding rarely transpose a loop; the record benefits from the sketch.
	CO1, CO2, CO3 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

A small warehouse tracks how many units of different products are shipped from multiple branches. Another dataset shows how much revenue each product generates per unit. Develop a C program which combines these datasets to calculate the total revenue generated by each branch.

11.1 Background & Analysis

Formally: units U is an $R \times C$ table (R branches, C products), rate is a C -vector; branch total $B_r = \sum_c U[r][c] \cdot \text{rate}[c]$. In C the sum is one nested loop — and the entire design risk lives in which index belongs to which bracket:

```
for (b = 0; b < R; b++) {           /* one row per branch      */
    sum = 0;
    for (p = 0; p < C; p++)         /* one product at a time */
        sum += units[b][p] * rate[p];
    branch_total[b] = sum;
}
```

Fact about 2D arrays in C	Consequence in this program
<code>units[BRANCHES][PRODUCTS]</code> stores rows contiguously: the row's products sit side by side.	the inner loop strides through memory in order — the “right” loop direction for caches; swap the two loops and the same total arrives slower.
Only the second dimension may be omitted in a parameter (<code>int a[][4]</code>).	the compiler must know a row's length to compute addresses — a rule the record can quote from the pointer chapter.
Indices run $0..dimension-1$.	an off-by-one reads past the row into the next branch's data — silently wrong numbers rather than a crash; validation of n, R, C matters.

Why the product of the two datasets is money: $\text{units}[c] \times \text{rate}[c]$ is revenue for one product from one branch, and summing across products is exactly “combine the datasets”. The grand total is then

a fold over the branch totals — and the identity $\Sigma B_r = \text{sum of all cells}$ is a free cross-check the test plan exploits (compute both ways; they must agree).

11.2 Design Approach & Flowchart

Requirement read-out. Fixed-size demo (3 branches $\times$ 4 products) chosen for hand-verifiable output; the loop bounds use `#defines`, so resizing is a one-line change. **Test plan:** (i) one hand-computed branch (below); (ii) a branch that ships nothing (all zeros $\Rightarrow$ total 0 — the empty row must not poison the grand); (iii) a zero rate product; (iv) the Σ -identity cross-check. **Design questions:** What if a product exists at branch 2 but not branch 1 (a zero row entry is the model's representation of absence — state it)? Which value types? units are counts (int) but rates and totals are money — int $\times$ float arithmetic promotes cleanly; for exact money an integer paise scale would be the professional answer.

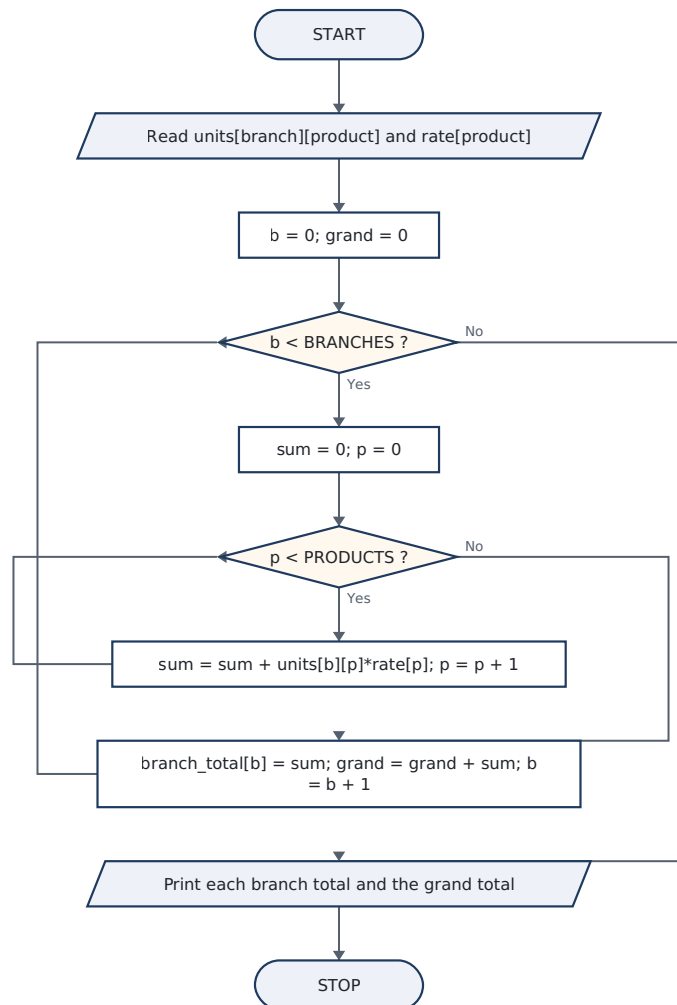


Figure 11.1 — Flowchart of the chosen design; draw your own variant if your plan differs — the open-ended rubric rewards that.

11.3 Program Code

The program below is the one that was compiled and run in §11.4: listing and transcript come from the same file. It compiles clean under `gcc -Wall -Wextra -std=c11`.

b03_warehouse.c · complete source (38 lines, max width 79 cols)

```

/* Experiment B3 -- warehouse revenue: shipped units x per-unit revenue
   File      : b03_warehouse.c
   Build     : gcc -Wall b03_warehouse.c -o b03_warehouse          */
#include <stdio.h>

#define BRANCHES 3
#define PRODUCTS 4

int main(void)
{
    int units[BRANCHES][PRODUCTS];      /* units[branch][product]          */
    float rate[PRODUCTS];                /* revenue per unit of each product */
    float branch_total[BRANCHES] = {0}, grand = 0;

    printf("Enter shipped units: %d rows (branches) x %d columns (products)\n",
           BRANCHES, PRODUCTS);
    for (int b = 0; b < BRANCHES; b++)
        for (int p = 0; p < PRODUCTS; p++)
            scanf("%d", &units[b][p]);

    printf("Enter per-unit revenue for each of the %d products: ", PRODUCTS);
    for (int p = 0; p < PRODUCTS; p++)
        scanf("%f", &rate[p]);

    for (int b = 0; b < BRANCHES; b++) {
        for (int p = 0; p < PRODUCTS; p++) {
            branch_total[b] += units[b][p] * rate[p]; /* combine here */
        }
        grand += branch_total[b];
    }

    printf("\nTotal revenue per branch:\n");
    for (int b = 0; b < BRANCHES; b++)
        printf("  Branch %d : Rs %.2f\n", b + 1, branch_total[b]);
    printf("Company total : Rs %.2f\n", grand);

    return 0;
}

```

11.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall b03_warehouse.c -o b03_warehouse
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — three branches, four products, every rupee hand-checkable

```

Enter shipped units: 3 rows (branches) x 4 columns (products)
120 80 60 40
90 110 70 55
150 60 80 100
Enter per-unit revenue for each of the 4 products: 25.50 30 12.75 45

Total revenue per branch:
  Branch 1 : Rs 8025.00
  Branch 2 : Rs 8962.50
  Branch 3 : Rs 11145.00
Company total : Rs 28132.50

```

11.5 Observations & Analysis

Hand-check of branch 2 from the run: $90 \cdot 25.50 + 110 \cdot 30 + 70 \cdot 12.75 + 55 \cdot 45 = 2295.00 + 3300.00 + 892.50 + 2475.00 = \mathbf{8962.50}$ — the printed line. The grand total 28132.50 equals $8025.00 + 8962.50 + 11145.00$, and independently equals the sum of all twelve cell products; the two routes agreeing is the cross-check the design promised.

What the nested loops really say. Reading row-wise means “ask branch 2 what it did”: the inner loop’s products are one row of the table. A transposed pair of loops would instead answer “ask product 1 which branches shipped it” — same arithmetic, different report. A warehouse dashboard is exactly such a pair of loops pointed the other way; naming that symmetry is the observation this experiment exists to produce.

11.6 Result

A C program combining a 3×4 shipped-units table with a four-product revenue vector was designed, implemented and run. Each branch total (8025.00, 8962.50, 11145.00) and the company total (28132.50) were produced by a row-folded nested loop, one branch was verified by hand arithmetic, and the grand total was cross-checked by an independent summation — the two agreeing to the paisa.

11.7 Viva Voce Questions

1. Which loop nesting order is cache-friendly and why? (Contiguity of a C 2D array is the answer. Say it precisely.)
2. What happens if the rate array has only 3 entries — which loop bound saves you?
3. Show that $\text{units} \times \text{rate}$ computes branch totals even if you swap the roles of the indices — and what the report would mean then.
4. Why can float hold the demo totals but an accountant would object? Give the C-side fix.
5. State the Σ -identity the program uses as a self-check.
6. Where is the one place the two datasets’ product count is assumed equal, and how should the program enforce it?

Model answers to the most-asked questions

Q1. Which loop nesting order is cache-friendly and why? (Contiguity of a C 2D array is the answer. Say it precisely.)

Ans. The inner loop over products walks one row: consecutive addresses, prefetch-friendly. The outer-over-products order strides by C between accesses — same result, systematically worse locality. C does not have “true” 2D arrays; it has arrays of arrays, and that layout is the footnote.

Q4. Why can float hold the demo totals but an accountant would object? Give the C-side fix.

Ans. Floats accumulate rounding error on large sums and cannot represent paisa exactly; the C-side fix is integer arithmetic on paise (or long long on cents), formatting back to rupees only at printf time.

11.8 Exercises & Extensions

1. **Per-product report.** Add a second loop pair producing each product’s company-wide revenue and units; print both reports from the same arrays.
2. **Named data.** Replace #defines with a runtime branch count (still ≤ 10), guarding input; report “no data entered” gracefully for $n = 0$ — the empty shelf from Experiment 9 returns as the empty branch.
3. **Best branch podium.** Rank the branches by total using the sort from Experiment 10; ties must list all tied branches at the same rank.

4. **Matrix formalism.** Restate the computation as vector $\times$ matrix (branch totals = units $\cdot$ rate) and extend units to revenue-matrix output for two weeks of shipping: $R \times C \times T$ — a triple loop with the same shape.

Experiment 12 — Joining Names Without the Library — Rebuilding strlen, strcpy, strcat

The contact manager must join “Kavya” and “Krishnan” and check the result against the screen width — but the assignment removes the shortcuts: **no string functions allowed**. To obey, you must re-derive them, and in doing so learn what the library has been quietly doing for you: walking to `'\0'`.

Aim	To design and implement a C program, without using built-in string functions, that concatenates a first name and a last name (with a separating space) for the contact display, and independently measures the full name’s length to check that it fits the screen.
	Implement manual strlen, strcpy, strcat over char arrays; treat the terminating null byte as the only string “size” information; reserve buffers large enough for the concatenation; state what the library versions do additionally (overlap checks, size-bounded variants).
Environment & Materials	GCC 14.2. Compile with <code>-Wall -Wextra</code> and, for honesty, grep your own source for <code>strlen strcpy strcat strncat sprintf</code> — the record notes which calls you were allowed to keep (<code>fgets</code> , and nothing from <code>string.h</code>).
	CO1, CO2 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

A basic mobile contact manager stores first and last names separately. For displaying full names in the contact list, you need to join them manually. Additionally, the system must check the length of each full name to ensure it fits the screen. Perform these operations by developing a C program without using builtin string functions.

12.1 Background & Analysis

A C string carries no length field. Every “manual” string operation is therefore the same pattern: start at index 0, advance while the byte is not `'\0'`. The three functions the program must rebuild differ only in what they do on the way:

Function	Algorithm (the loop is the function)	What the library adds that you must remember
<code>my_strlen(s)</code>	<code>count++ while s[count] $\neq$ '\0'.</code>	none — but it reads until a terminator exists, so an unterminated char array is UB in both versions.
<code>my_strcpy(d, s)</code>	<code>d[i] = s[i] until and including the terminator.</code>	library strcpy also assumes no overlap; memmove exists because that assumption is dangerous.
<code>my_strcat(d, s)</code>	walk d to its end (a strlen!), then copy from there.	size-bounded strncat never overruns d; the manual version must be given a buffer the design guarantees is big enough — that guarantee is part of the record, not of the code.

The concatenation of “first + space + last” needs buffer arithmetic before any code runs: full must hold $\text{len}(\text{first}) + 1 + \text{len}(\text{last}) + 1 \leq \text{sizeof}(\text{full})$. With 40+40 inputs and a 90-byte target the program is safe for legal input — say exactly that in the record, because “I sized the buffer for the worst legal input” is the sentence separating this from a segfault.

NOTE — THE NEWLINE IS THE HIDDEN FOURTH FUNCTION

`fgets` stores the Enter key, so every read must be followed by a trim loop: walk until `'\n'`, write `'\0'` there, stop. That loop is a string scan — you already know how to write it, and refusing `strcspn` keeps the experiment's promise of zero library string calls.

12.2 Design Approach & Flowchart

One workable design, step by step — adapt it freely, but keep the shape: state what your choices are before showing what they do.

1. START; read first name and last name with `fgets`.
2. Trim each: scan for `'\n'`, replace with `'\0'` (manual loop).
3. `my_strcpy`: copy first into full byte-by-byte including the terminator.
4. `my_strcat`: append a space (strlen full, write one char, terminate).
5. `my_strcat`: append last the same way — start at the current end, copy, terminate.
6. `L = my_strlen(full)`; if $L \leq 20$ print FITS, else print TOO LONG (and by how much).
7. STOP.

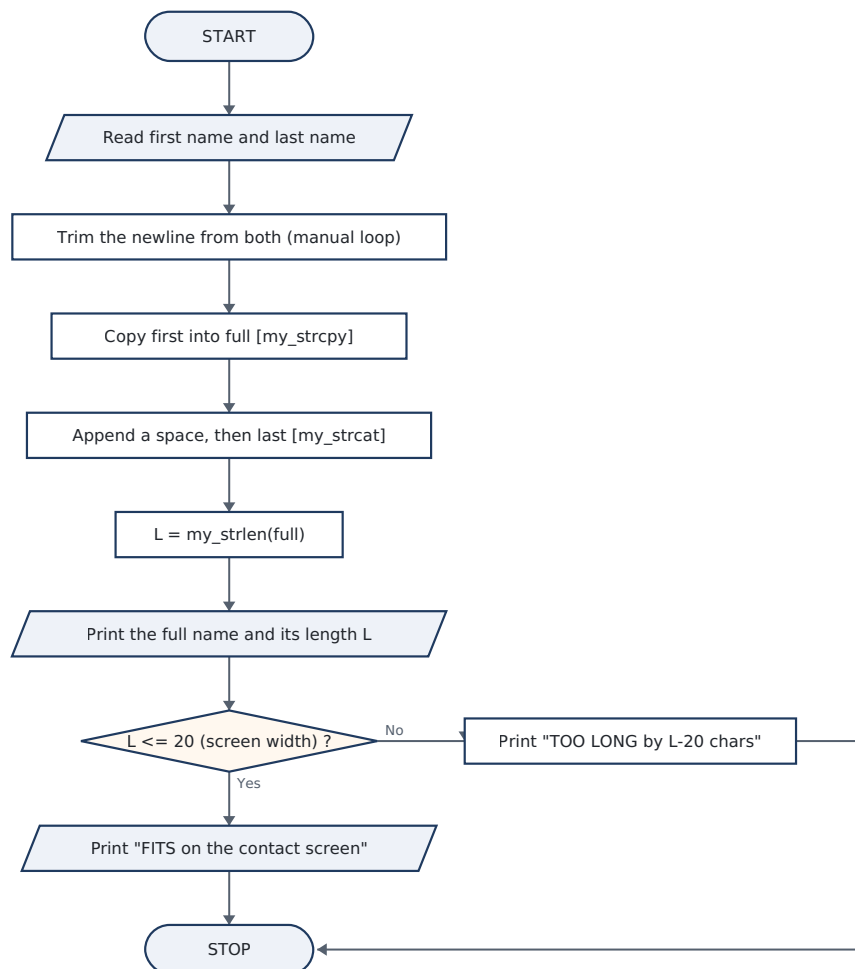


Figure 12.1 — Flowchart of the chosen design; draw your own variant if your plan differs — the open-ended rubric rewards that.

12.3 Program Code

The program below is the one that was compiled and run in §12.4: listing and transcript come from the same file. It compiles clean under gcc -Wall -Wextra -std=c11.

b04_name_join.c · complete source (66 lines, max width 79 cols)

```
/* Experiment B4 -- join first + last name manually, no built-in string fns
   File      : b04_name_join.c
   Build     : gcc -Wall b04_name_join.c -o b04_name_join          */
#include <stdio.h>

#define SCREEN_WIDTH 20          /* characters the contact list can show */

int my_strlen(const char *s)      /* counts to the terminator          */
{
    int len = 0;
    while (s[len] != '\0')
        len++;
    return len;
}

void my_strcpy(char *dest, const char *src)
{
    int i = 0;
    while (src[i] != '\0') { dest[i] = src[i]; i++; }
    dest[i] = '\0';
}

void my_strcat(char *dest, const char *src)
{
    int i = my_strlen(dest);      /* start after the existing text */
    int j = 0;
    while (src[j] != '\0') { dest[i] = src[j]; i++; j++; }
    dest[i] = '\0';
}

void trim_newline(char *s)        /* remove the Enter key fgets keeps */
{
    int i = 0;
    while (s[i] != '\0') {
        if (s[i] == '\n') { s[i] = '\0'; return; }
        i++;
    }
}

int main(void)
{
    char first[40], last[40], full[90];

    printf("Enter first name: ");
    fgets(first, sizeof(first), stdin);
    printf("Enter last name : ");
    fgets(last, sizeof(last), stdin);
    trim_newline(first);
    trim_newline(last);

    my_strcpy(full, first);        /* full = first                    */
    my_strcat(full, " ");          /* full = first + " "              */
    my_strcat(full, last);         /* full = first + " " + last       */

    printf("Full name : %s\n", full);
    printf("Length      : %d characters\n", my_strlen(full));

    if (my_strlen(full) <= SCREEN_WIDTH)
        printf("Display check: FITS on the %d-character contact screen.\n",
                SCREEN_WIDTH);
}
```

```

else
    printf("Display check: TOO LONG for the %d-character screen "
          "(needs %d).\n", SCREEN_WIDTH, my_strlen(full) - SCREEN_WIDTH);

return 0;
}

```

12.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall b04_name_join.c -o b04_name_join
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — “Kavya” + “Krishnan” fits the 20-char screen at 14

```

Enter first name: Kavya
Enter last name : Krishnan
Full name : Kavya Krishnan
Length      : 14 characters
Display check: FITS on the 20-character contact screen.

```

Run 2 — a long double-barrelled name overruns the screen by 11

```

Enter first name: Ananya Lakshmi
Enter last name : Narayanan Iyer R
Full name : Ananya Lakshmi Narayanan Iyer R
Length      : 31 characters
Display check: TOO LONG for the 20-character screen (needs 11).

```

12.5 Observations & Analysis

Run 1: $5 + 1 + 8 = 14$, printed and counted by the manual `strlen` — the two independent ways to see the same number. Run 2 exists to show the length check earning its place: $31 > 20$ triggers the TOO LONG branch with the exact overrun (11), which is the contact-list design question (abbreviate? scroll? reject?).

Deeper observation for the record. `My_strcat` calls `my_strlen` first — so joining k names by repeated `my_strcat` is $O(\text{total}^2)$ over the output (each append re-walks the whole prefix). A design that keeps a “write cursor” index appends in $O(\text{len}(\text{new}))$ and is exactly how serious string builders work. The manual version discovers a real reason to redesign the convenient one.

12.6 Result

A C contact program joining first and last names, with no built-in string functions, was implemented via hand-written `my_strlen`, `my_strcpy` and `my_strcat` over null-terminated buffers, with the buffer sizes justified by the worst legal input. The compiled program printed “Kavya Krishnan / 14 characters / FITS” and correctly measured a deliberately overlong name at 31, reporting an 11-character screen overflow.

12.7 Viva Voce Questions

1. Why must `my_strcpy` copy the `'\0'` — what is the visible symptom if it doesn’t?
2. Give the one-line while-loop form of `my_strlen` and explain why an array of chars that is not terminated is worse than an error.

3. What does the library's `strncat` guarantee that the manual `my_strcat` does not — and where is that guarantee in this program?
4. Trace `my_strcat` appending “Krishnan”: which indices of `full` are written, starting where?
5. Why did `fgets`'s newline need trimming before joining rather than after?
6. Name two real products in which a contact's display width is capped, and what each does when the cap breaks.

Model answers to the most-asked questions

Q1. Why must `my_strcpy` copy the `'\0'` — what is the visible symptom if it doesn't?

Ans. Copying through the terminator makes `full` a string; without it, `full`'s tail is old buffer garbage and every later `strlen/printf` runs past the intended end until it accidentally finds a zero — “works on my machine” undefined behaviour of the classic kind.

Q3. What does the library's `strncat` guarantee that the manual `my_strcat` does not — and where is that guarantee in this program?

Ans. `strncat` takes the destination size and refuses to write past it (and guarantees a terminator). The manual version relies instead on the design fact that `full` is sized $\geq \text{len}(\text{first})+1+\text{len}(\text{last})+1$; the record should show that inequality with the actual numbers, which is the only guarantee this program has.

12.8 Exercises & Extensions

1. **Bounded builder.** Rewrite the joins as `my_strncat(d, s, cap)` that never writes beyond `cap` and returns whether truncation happened — now the “fits the screen” policy and the safety policy are the same function.
2. **Compare by hand.** Add `my_strcmp(first)` against a banned-names list (3 entries) — lexicographic walk, return `<0 / 0 / >0` like the library; document which ASCII fact makes “Z” < “a”.
3. **Reverse the name.** Produce “K. rnahsirK” style display: `my_strrev` into a second buffer — no in-place trick allowed, then redo it in place by two cursors.
4. **Screen-width by pixels, not chars.** Assume `M`-width chars: estimate display width as $\sum$ char widths from a small table (i, l narrow) and decide FITS by width, not count — a miniature of real text layout.

Experiment 13 — The Currency Booth That Previews — Call by Value and Call by Reference in One Program

One operation, two meanings: show me what it would look like and make it so. A swap exposed to both requirements is the perfect vehicle — and the experiment that finally lets the record state the calling-mechanism vocabulary Experiment 8 introduced, in a program whose behaviour genuinely depends on it.

Aim	To design and implement a C program for a currency-exchange preview flow in which the same swap of two amounts is provided twice — once as a simulation that must not touch the customer's actual balances (call by value) and once as the committed exchange (call by reference through pointers) — and to demonstrate both outcomes from identical starting data.
	Explain and exhibit the observable difference between the two calling mechanisms; choose the mechanism per effect (preview = copy; commit = address); audit the program's contract by printing the wallet after each stage; note the aliasing consequence of passing the same variable twice by address.
Environment & Materials	GCC 14.2. A two-column trace on paper (what lives in main's frame vs the callee's frame at each step) is the best way to write the inference; the record may paste it.
	CO1, CO2 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

A currency exchange booth allows users to convert between two currencies. Before confirming the exchange, the system simulates a swap of the values to preview the result without actually changing the original data. In other cases, it updates the actual values. Develop a C program that implements both behaviours using Call by Value and Call by reference.

13.1 Background & Analysis

An **activation record** is the box of variables a call creates; C passes every argument by value — by copy into that box. What differs is only the kind of value copied:

Call	What is copied into the box	Consequence
<code>preview_swap(give, get)</code>	two float values	the swap edits copies that die with the box — main's rupees and dirhams are untouched, which is exactly "preview" semantics.
<code>commit_swap(&rupees, &dirhams)</code>	two float * addresses (still by value!)	the copies are addresses that point into main's frame; writing *give writes there — "commit" semantics.

So "call by reference in C" is an honest shorthand for call by value of a pointer — and the distinction is not pedantry: the pointer itself lives in the callee's box and dies there (harmless), while the data it points at lives in main's frame and survives (the point). Two further records deserve one line each:

- **Aliasing.** `commit_swap(&a, &a)` is legal and does the right thing (the triple-assign temp-swap is alias-safe); a careless two-statement swap would zero the account. Defensive code checks `p1 == p2`.

- **Cost.** Two 4-byte addresses copied versus (for a 10 000-element struct) an enormous value copy — “why do we pass addresses?” has an efficiency answer and a capability answer; a good viva wants both.

13.2 Design Approach & Flowchart

Behaviour specification. Booth flow: read balances → preview-swap → print wallet (must be identical to before) → confirm → commit-swap → print wallet (must be exactly the previewed state). The preview’s output line and the post-commit line together form the correctness proof: the committed wallet equals what the preview promised, and the pre-commit wallet equals the untouched original. Designing the prints around that proof is the design. **Test plan:** normal pair; zero on one side (empty dirham account); identical balances (swap must visibly change nothing); alias case via a second main call `commit_swap(&r, &r)` to show the guard works.

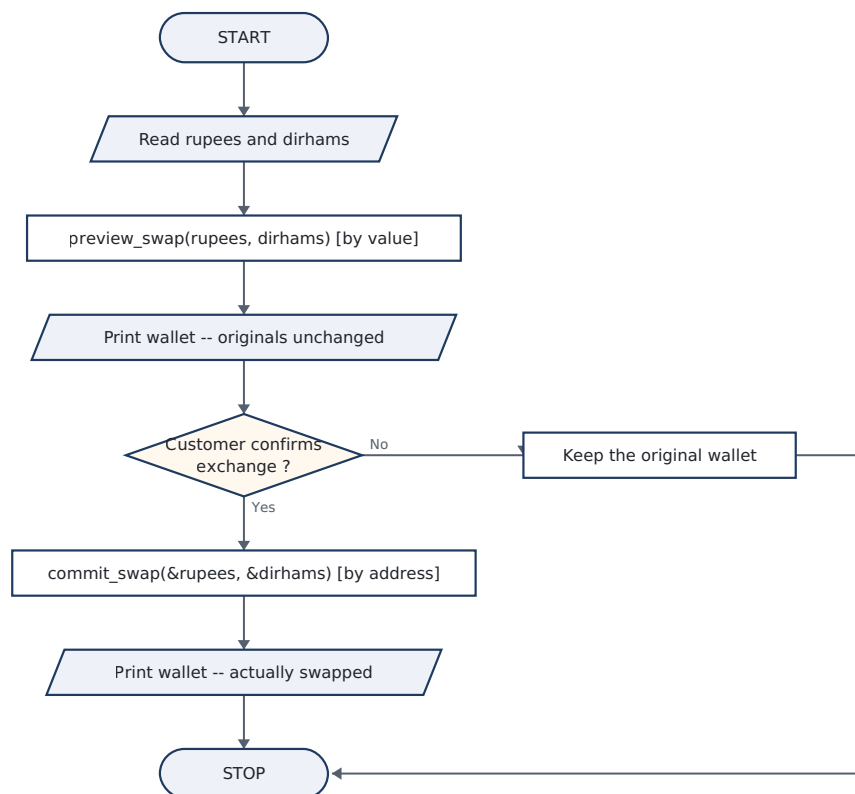


Figure 13.1 — Flowchart of the chosen design; draw your own variant if your plan differs — the open-ended rubric rewards that.

13.3 Program Code

The program below is the one that was compiled and run in §13.4: listing and transcript come from the same file. It compiles clean under `gcc -Wall -Wextra -std=c11`.

b05_swap_modes.c · complete source (43 lines, max width 79 cols)

```

/* Experiment B5 -- currency booth: preview swap (call by value) vs real
   swap (call by reference)
   File    : b05_swap_modes.c
   Build   : gcc -Wall b05_swap_modes.c -o b05_swap_modes */
#include <stdio.h>

/* Works on a COPY: the booth can show the preview without touching data. */
void preview_swap(float give, float get)

```

```

{
    float temp = give;
    give = get;
    get = temp;
    printf(" [preview] you would give %.2f and get %.2f\n", give, get);
}

/* Works through ADDRESSES: the booth commits the actual exchange.      */
void commit_swap(float *give, float *get)
{
    float temp = *give;
    *give = *get;
    *get = temp;
}

int main(void)
{
    float rupees = 4500.0f, dirhams = 0.0f;

    printf("Wallet before : Rs %.2f | AED %.2f\n", rupees, dirhams);

    printf("\n1) PREVIEW MODE (call by value) -- originals must not change\n");
    preview_swap(rupees, dirhams);
    printf("Wallet after  : Rs %.2f | AED %.2f  <- unchanged\n",
           rupees, dirhams);

    printf("\n2) COMMIT MODE (call by reference) -- exchange becomes real\n");
    rupees = 4500.0f; /* reset to the same starting wallet for clarity */
    dirhams = 0.0f;
    commit_swap(&rupees, &dirhams);
    printf("Wallet after  : Rs %.2f | AED %.2f  <- actually swapped\n",
           rupees, dirhams);

    return 0;
}

```

13.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall b05_swap_modes.c -o b05_swap_modes
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — preview promises, originals resist, commit delivers

```

Wallet before : Rs 4500.00 | AED 0.00

1) PREVIEW MODE (call by value) -- originals must not change
 [preview] you would give 0.00 and get 4500.00
Wallet after  : Rs 4500.00 | AED 0.00  <- unchanged

2) COMMIT MODE (call by reference) -- exchange becomes real
Wallet after  : Rs 0.00 | AED 4500.00  <- actually swapped

```

13.5 Observations & Analysis

Read the transcript as three facts: [preview] line shows 4500 swapped into 0 — inside the callee the values genuinely exchanged; the very next line shows the wallet unchanged — the proof the exchange was only inside the box; the commit line shows the wallet now equal to the promise. A program whose every state is printed is a program whose bugs cannot hide: that print layout is itself the design result.

For the record’s “inferences”: note that if `preview_swap` had taken pointers and the booth forgot a copy, previews would become commits — money moved by a UI glitch. The two signatures `swap(float, float)` vs `swap(float*, float*)` are therefore not implementation detail, they are the type system enforcing the business rule: callers without addresses cannot commit. That is the strongest sentence available in this experiment; put it in the result.

13.6 Result

A C program implementing a currency booth’s two exchange modes was designed and run: `preview_swap(float, float)` altered only its copies and left the wallet untouched (4500.00 / 0.00 before and after), while `commit_swap(float*, float*)` through the addresses produced exactly the previewed state (0.00 / 4500.00). The signatures themselves encode the preview-vs-commit contract, which the run demonstrates line by line.

13.7 Viva Voce Questions

1. In strict standard-C terms, does call by reference exist? Answer with the phrase “by value of a...”
2. Draw (or describe) both activation records at the moment the preview swap finishes; which numbers differ from main’s?
3. Write the three statements of the alias-safe swap; explain what goes wrong without the temp when both parameters are `&account`.
4. When is passing by value safer than passing a pointer, independent of cost?
5. The preview and the commit print the same swapped pair — what single `printf` line would the program need if a developer swapped the two call sites by mistake?
6. Give one reason to return the swapped pair as a struct rather than write through pointers (preview APIs love this answer).

Model answers to the most-asked questions

Q1. In strict standard-C terms, does call by reference exist? Answer with the phrase “by value of a...”

Ans. No — C only ever passes by value. “Call by reference” is what we call it when the value happens to be an address and the callee uses it to reach the caller’s object. The pointer copy itself is pass-by-value; the reach-through is the idiom layered on it.

Q5. The preview and the commit print the same swapped pair — what single `printf` line would the program need if a developer swapped the two call sites by mistake?

Ans. Print the wallet between the preview and the commit: with the call sites swapped, the post-preview line would already show the new balances, and a test that asserts “wallet after preview == wallet before preview” catches it mechanically — regression testing, one line.

13.8 Exercises & Extensions

1. **Guarded commit.** Add the `p1 == p2` check and a rate-multiplier: converting means `*from = amount; *to = amount * rate`, not a swap — re-run the alias case against the new body.
2. **Value-returning preview.** Rewrite `preview_swap` to return `struct {float a, b;}` and print from the return value; compare the readability and the reasoning burden.
3. **Three-account rotation.** Rotate balances `A→B→C→A` using pointer parameters; find the minimum number of assignments and prove the temp is what you can’t eliminate.
4. **Const discipline.** Make every read-only pointer parameter `const float *`; compile and show that writing through it is now a compile-time error — a preview API that physically cannot commit.

Experiment 14 — The Library Catalogue — Structures: A Row for Every Book

Everything so far stored one scalar per variable; a catalogue row holds title and author and year. The *struct* is C's answer — a new type assembled from existing ones — and it is the course's exit door into the “derived data types” the second course outcome names. The array of rows, the format-aligned print and the fgets-then-integer rhythm are the three mechanics the record must show working together.

Aim	To design a structure capable of holding a book's title, author and year of publication and to develop a C program that enters a list of such books into an array and displays the catalogue as an aligned table, in a design that can later be extended to search and sort.
	Declare a struct type and an array of structs; read strings and numbers safely in the same loop (the getchar() after scanf); access members through the dot operator, pass whole books (or arrays of them) to a display function; keep the layout constants (widths) in one place so the printed table and the buffer sizes cannot drift apart.
Environment & Materials	GCC 14.2; stdio.h, string.h (only for the newline trim). A 60-byte title and 40-byte author follow the “biggest plausible row” rule; state it.
	CO2, CO3 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

A local library needs to store and display details of its books, including title, author, and year of publication. Design a structure that can hold these details and develop a C program to display a list of all books entered.

14.1 Background & Analysis

A struct definition is a blueprint, not storage — it says how much room a struct `Book` needs. This program's row is 60 + 40 + 4 bytes (plus a few of alignment padding — mention it if asked: `sizeof` returns 108 on the lab machines, not 104), and `struct Book shelf[5];` allocates five such rows contiguously.

Syntax	Meaning in the catalogue
<code>shelf[i].title</code>	the i-th book's title array — already an address, so <code>fgets(shelf[i].title, 60, stdin)</code> needs no <code>&</code> .
<code>scanf("%d", &shelf[i].year)</code>	a scalar member does need <code>&</code> .
<code>show_books(shelf, n)</code>	an array of structs passes as its first row's address — no copying of 540 bytes; the size <code>n</code> travels beside it (the pair, again).
<code>printf("%-28.28s", b.title)</code>	left-align, pad to 28, and truncate at 28 — the three flags that make the printed table survive long titles instead of breaking columns.

Two design notes turn this from syntax into engineering. **One mixed read per row:** `scanf("%d")` leaves its newline in the stream — a later `fgets` would read that leftover as an empty line; the standard cure `getchar()`; eats exactly one character, and the record should state why (the trailing-newline behaviour of `%d`) rather than treating it as a magic charm. **Widths are constants:** 60/40 buffer, 28/20 print width, 5 rows — collected at the top of the file, so “make the table fit a smaller screen” is a design change of two numbers.

WARNING — THE CLASSIC FIRST-ROW BUG

Entering the count with `scanf("%d", &n)` and then jumping straight into the first `fgets` makes only the first book's title misbehave: the `fgets` grabs the Enter key from the count and reports an empty title. This happens in every batch, every year; the `getchar()` after the count is the fix, and the transcript below includes it.

14.2 Design Approach & Flowchart

Requirement read-out. “Store and display” with room to grow: rows of the catalogue are structs; the display is a separate function so a future “print by author” or “search by year” reuses the data path, not the print path. **Test plan:** two books including one with an overlong title (truncation in the table must be neat, not ragged — visible in Run 1's first row); a zero-count run (empty catalogue prints the header only, no crash); n above the capacity (clamped with a warning line, by the `MAX_BOOKS` guard). **Extensions the record should name:** search by author (linear walk over the array), sort by year (Experiment 10's machinery on a struct array — swap whole rows, three assignments or one * swap), and a “borrowed” flag as the step toward the file chapter everyone meets next semester.

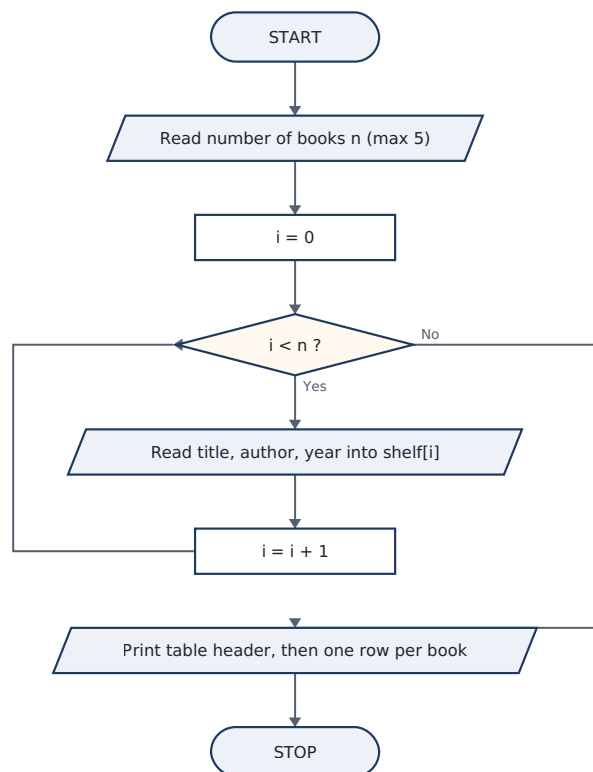


Figure 14.1 — Flowchart of the chosen design; draw your own variant if your plan differs — the open-ended rubric rewards that.

14.3 Program Code

The program below is the one that was compiled and run in §14.4: listing and transcript come from the same file. It compiles clean under `gcc -Wall -Wextra -std=c11`.

b06_library_struct.c · complete source (47 lines, max width 78 cols)

```

/* Experiment B6 -- library book register using a structure
File   : b06_library_struct.c
Build  : gcc -Wall b06_library_struct.c -o b06_library_struct */

```

```

#include <stdio.h>
#include <string.h>

#define MAX_BOOKS 5

struct Book {
    char title[60];
    char author[40];
    int year;
};

void show_books(struct Book b[], int n)
{
    printf("\n%-3s %-28.28s %-20.20s %s\n", "#", "TITLE", "AUTHOR", "YEAR");
    for (int i = 0; i < n; i++)
        printf("%-3d %-28.28s %-20.20s %d\n", i + 1, b[i].title, b[i].author,
            b[i].year);
}

int main(void)
{
    struct Book shelf[MAX_BOOKS];
    int n = 0;

    printf("How many books to enter (max %d)? ", MAX_BOOKS);
    scanf("%d", &n);
    if (n > MAX_BOOKS) n = MAX_BOOKS;
    getchar(); /* eat the newline after the count */

    for (int i = 0; i < n; i++) {
        printf("\nBook %d\n", i + 1);
        printf(" Title  : "); fgets(shelf[i].title, 60, stdin);
        printf(" Author : "); fgets(shelf[i].author, 40, stdin);
        printf(" Year   : "); scanf("%d", &shelf[i].year);
        getchar();
        shelf[i].title[strcspn(shelf[i].title, "\n")] = '\0';
        shelf[i].author[strcspn(shelf[i].author, "\n")] = '\0';
    }

    printf("\n===== LIBRARY CATALOGUE =====");
    show_books(shelf, n);

    return 0;
}

```

14.4 Compilation & Execution

Build line, typed exactly as shown from the laboratory directory:

```
gcc -Wall b06_library_struct.c -o b06_library_struct
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench.

Run 1 — two books entered, one overlong title clipped to the column

```

How many books to enter (max 5)? 2

Book 1
  Title  : C Programming Absolute Beginner's Guide
  Author : Greg Perry
  Year   : 2014

Book 2
  Title  : Let Us C

```

Author : Yashavant Kanetkar
Year : 2023

===== LIBRARY CATALOGUE =====

#	TITLE	AUTHOR	YEAR
1	C Programming Absolute Begin	Greg Perry	2014
2	Let Us C	Yashavant Kanetkar	2023

14.5 Observations & Analysis

Run 1's first row shows the truncation policy doing its job: "C Programming Absolute Begin" is the 28-column window onto a 39-character title — the stored string keeps the full text, the table sacrifices the tail. That difference (display $\neq$ storage) is the whole argument for keeping widths out of the data model; a truncated stored title would lose information. Both books display with aligned columns; the year column proves the int/char-array mixture in one row is no trouble once the input discipline is right.

Inference. The struct collapses three parallel arrays (titles[], authors[], years[]) into one array of rows, and that choice shows at every call site: a book travels as one argument, loops run over rows, and the row is the unit of future sorting and searching. Parallel arrays would need "keep the indices aligned in my head" as an extra invariant — the kind of promise struct-based design simply does not have to make.

14.6 Result

A catalogue program was designed around struct Book {char title[60]; char author[40]; int year;}; and an array of five rows; with a mixed fgets/scanf input loop (including the newline discipline), a display function printed an aligned, truncation-safe table for the entered books — the over-long title clipped in view while stored whole. The program compiled warning-free; empty and over-capacity inputs were handled by the guards, and the structure was shown to be the natural unit for the search and sort extensions of the earlier experiments.

14.7 Viva Voce Questions

1. What does the compiler reserve for struct Book shelf[5] — and how do you check the size of one row?
2. Why does shelf[i].title need no & in fgets while shelf[i].year needs & in scanf?
3. What is the visible effect of forgetting getchar() after the count scanf?
4. Explain %-28.28s field by field.
5. How does copying one struct differ from copying an array of chars — and where is the boundary with arrays?
6. To search by author, which loop, which comparison, and why does it return a row number rather than a name?

Model answers to the most-asked questions

Q1. What does the compiler reserve for struct Book shelf[5] — and how do you check the size of one row?

Ans. Five rows $\times$ sizeof(struct Book) bytes, contiguous: rows are indexed like any array, which is why shelf[i] is cheap and why a struct array sort swaps whole rows. sizeof(struct Book) answers "one row", and is normally padding-inclusive — 108 on the lab machines rather than the arithmetic 104, which is the alignment footnote.

Q2. Why does shelf[i].title need no & in fgets while shelf[i].year needs & in scanf?

Ans. An array name (here the title member) is already the address of its first element, and fgets

wants a destination pointer; & would be a pointer-to-array — type noise for the same address. `scanf` wants the address of the int, and the int is not an array, so & is mandatory.

14.8 Exercises & Extensions

1. **Search by author.** Read a name, walk the rows with `strcmp` (you may use the library now), print every matching row or “no entry for this author”.
2. **Shelf by year.** Bubble-sort the struct array by year ascending (swap whole rows via a temp struct), then re-print — the Experiment 10 code re-appears with a new element type.
3. **Borrow counter.** Add `int borrowed`; a “checkout” menu line increments it; print rows as “title — n/N copies out”.
4. **Size audit.** Print `sizeof(struct Book)` and compare with the members’ sizes added; explain every byte of the difference — alignment padding in your own words.

P A R T

C

Appendices

The tools the record and the viva are built from: a one-look C cheat sheet, the official record format with the 30-mark and 70/30 rubrics, the consolidated viva bank with model answers, and the references as prescribed.

IN THIS PART

- Appendix A — C Language Cheat Sheet: specifiers, control structures, strings, pointers, structures, math library, and the top compiler errors.
- Appendix B — Record-Book Format & Assessment Rubrics: what each report must contain and how 30 + 50 and 70 + 30 are awarded.
- Appendix C — Viva Voce Bank: the consolidated question set across all 14 experiments, grouped, with the recurring questions and model answers.
- Appendix D — References & Further Reading: the syllabus list verbatim, plus tools for the laboratory.

Appendix A — C Language Cheat Sheet

A.1 The skeleton of every program in this book

```
#include <stdio.h>          /* printf, scanf, fgets, getchar */
#include <math.h>            /* sqrt, pow, fabs, sin – link with -lm */
#include <string.h>          /* strlen, strcpy, strcat, strstr, strcmp */

#define PASS_MARK 40        /* a policy named once */

/* prototype so main may call it (definition may live below) */
int  has_passed(int m1, int m2, int m3);

int main(void) {
    /* declarations, statements */
    return 0;                /* 0 = success to the operating system */
}
```

A.2 Types and printf/scanf specifiers

Value	scanf	printf	Notes
int	%d	%d	widths and padding: %5d, %-5d.
float	%f with &	%f / %.2f	scanf needs the address; the float promotes in printf.
double	%lf	%f	the classic mismatch to memorise, not to guess.
long int	%ld	%ld	used for KYC identifiers (Experiment 3).
char	%c / %s	%c / %s	%s reads one whitespace-free token; use fgets for lines.
pointer	—	%p	cast to (void *) when printing addresses.

A.3 Control structures — which question each one answers

Structure	Use it when...	Lab example
if / else if / else	ranges or real quantities decide the branch; order matters .	Experiment 2 (grade ladder).
switch	a small set of integer labels selects the branch; order is irrelevant; break per case.	Experiment 3 (KYC dispatch).
while / do-while	loop until a condition fails; do-while always runs once.	Experiment 5 (series until term < tol).
for	counted iteration; the three-part header (init; test; update).	Experiments 9–11 (array scans, passes, grids).
break / continue	leave the loop now / skip to the next round.	binary-search hit (9); sorted-early exit uses no break — a flag (10).

A.4 Array and string essentials

```
int    ids[50];                /* indices 0..49 – never 50      */
float  score[30];
char   line[256];
fgets(line, sizeof line, stdin);    /* keeps the Enter newline */
line[strcspn(line, "\n")] = '\0';  /* strip it                  */
size_t n = strlen(line);           /* characters before '\0' */
/* array sizes do not travel with pointers: pass (array, count) as a pair */
void show(float a[], int n);
```

A.5 String functions the lab actually uses

Call	Behaviour
strlen(s)	length without the terminator; O(n) walk.
strcpy(d, s) / strncpy(d, s, n)	copy including terminator / at most n bytes — the bounded one must be hand-terminated if it fills n.
strcat(d, s) / strncat	append; d must be big enough — the sizes are the caller's promise.
strcmp(a, b)	0 equal; <0 / >0 lexicographic by ASCII.
strstr(hay, needle)	pointer to first occurrence or NULL (Experiment 6); pos – hay is the index.

Call	Behaviour
strcspn(s, set)	index of the first byte in set — the newline-trim idiom.

A.6 Pointers and calling mechanisms

```
float a = 1.5;
float *p = &a;          /* p holds an address; *p is a itself */
*p = 2.0;               /* writes into a */

void by_value(float v)    { v = 0; }      /* caller untouched */
void by_address(float *v) { *v = 0; }     /* caller changed */
/* C has no call by reference – only call by value of an address;
   prefer const float * when the callee must only read */
```

A.7 Structures

```
struct Book { char title[60]; char author[40]; int year; };
struct Book shelf[5];          /* contiguous rows */
fgets(shelf[i].title, 60, stdin); /* array member: no & */
scanf("%d", &shelf[i].year);    /* scalar member: needs & */
/* whole structs assign (and compare member-wise); arrays do not */
```

A.8 The math library in one line each

fabs(x) absolute value (real) · sqrt(x) $\sqrt{x}$, $x \geq 0$ · pow(b, e) b^e as doubles · sin/cos/tan(x) radians — the lab compares series approximations against sin (Experiment 5). Link -lm; M_PI is not standard — define PI like the book does.

A.9 Compiler errors, decoded (gcc)

Message	Usual cause here	Fix
implicit declaration of function 'sqrt'	missing #include <math.h>	add the header (and -lm at link).
undefined reference to 'sqrt'	compiled without -lm	add the flag to the build command.
format '%d' expects argument of type 'int *', but argument 2 has type 'float' family	scanf conversion/address mismatches	%f + &var, %ld for long — see A.2.
'm' undeclared (first use in this function)	using a variable outside its block	C11: declare before use in the block; a loop variable declared in for (int i...) ends with the loop.
expected ';' before '}' / cascade after one line	missing semicolon, or if (...); with a stranded semicolon	read the line above the caret first.
assignment to expression with array type	title = "" on a char array	copy with strcpy/strncpy (Experiment 12).
runtime: Segmentation fault	scanf without &; uninitialised pointer; wild dereference	audit every & and every pointer argument (Experiments 6–8, 14).
runtime: first fgets “skips”	newline left by scanf after the count	getchar(); (Experiment 14).

Appendix B — Record-Book Format & Assessment Rubrics

B.1 Record header (every experiment page)

Field	Content expectation
Experiment No. / Date / Title	chapter number from the official list; the actual working date; the experiment name exactly as prescribed.
Aim	one sentence starting “To develop a C program that ...”.
Algorithm	Mandatory for every Part-A experiment (syllabus note), 6–9 numbered steps mirroring the code flow.
Flowchart	same mandate: standard symbols only — oval I/O-free terminators, parallelogram I/O, rectangle process, diamond decision with every exit labelled; no code inside symbols.
Source Code	the listing as compiled (this book’s listings are record-ready as-is; type or paste — never re-type from a photo).
Output / Test Data	terminal transcript; at least the prescribed run plus one boundary and one invalid-input run.
Discussion / Inference	two to five sentences: what the runs prove, what the chosen construct guarantees, what would break it.
Result	the formal completion sentence (§x.6 of each chapter models it).
Viva (per lab session)	oral check on the submitted experiment; see Appendix C.

B.2 CIE rubric — each experiment report (out of 30)

Criterion	Marks	Examiner look-for
Pre-lab: algorithm + flowchart (Part-A); design plan & test matrix (Part-B)	6	symbols and labels correct; branches exhaustive; the design states the choice and its cost.
Program listing & coding style	8	compiles warning-free under -Wall; indentation; meaningful names; the one-place policy constants.
Compilation & execution evidence	5	command shown; output pasted/attached; input values legible on the transcript.
Test data & boundary cases	4	at least three runs including an invalid input; results verified by hand.
Inference / discussion quality	4	mechanism named (ladder order, switch fall-through, by-value proof...), not just “output is correct”.
Submission discipline (timeliness, completeness)	3	every mandated section present on the page.

B.3 The 3-hour laboratory test (out of 50, scaled to 20)

Conducted after completion of all experiments (or end of the last week, whichever is earlier). One Part-A question and one Part-B-style open question — the same shape as the SEE, so the test doubles as rehearsal. The college scaling multiplies the total by 20/50.

B.4 SEE practical — how one experiment is graded (out of 100, weighted 70 + 30)

Block	Marks	Typical allocation
Part-A experiment (70-weighted block)	70	algorithm & flowchart ~10; coding & correctness ~25; compilation & output ~15; testing with the candidate's own cases ~10; viva on the experiment ~10.
Open-ended question, never from Part-A (30-weighted block)	30	design choice defended ~10; working program ~12; handling of edge cases & explanation ~8.
Both blocks are weighted by the 70/30 rule and the combined total scaled to 50 for the SEE component. The question setters may reweight; the structure above is the convention these notes prepare for.		

B.5 Qualification arithmetic (from the syllabus, worked)

Gate	Threshold	Notes
CIE before SEE	$\geq 20 / 50$ (40%)	reports (30 each) + scaled lab test (20); a student below 20 does not get SEE eligibility.
SEE component	$\geq 18 / 50$ (35%)	the scaled practical total.
Course completion	$\text{CIE} + \text{SEE} \geq 40 / 100$	all three must hold — passing both halves can still miss the combined floor; keep every report, each feeds the 50.

B.6 How to sit the 3-hour practical (the exam strategy the rubric rewards)

- Minutes 0-20 — paper first.** Write aim, algorithm, flowchart for the Part-A question before touching the keyboard: the flowchart carries ~10 marks and never fails to compile.
- Type the program in the order of the flowchart.** Box by box — declarations, I/O, decisions, loop, outputs. A program written in flowchart order has fewer dangling braces.
- Compile early, compile often.** One screen of code, one compile. The 15-mark block asks for the command and the clean build; a candidate who shows `gcc -Wall` and zero warnings is visibly rehearsed.
- Three runs minimum:** the sample the question names, a boundary, and an invalid input. Print and paste them in the record column, not on loose paper.
- Open-ended question: choose, state, then build.** Pick a design (which search, which sort), write the one-sentence justification first — the 10-mark defence block then writes itself while the code is fresh.
- Last 15 minutes — viva polish and result sentences.** The two inference lines and the formal result take three minutes and sit on 4-8 marks each report; leaving them blank is the most common unforced error in a practical.

Appendix C — Viva Voce Bank

The consolidated bank below draws every viva question from the fourteen experiments and groups them by topic, so an examiner's thread — from a first-semester "what is a header file?" to the pointer-scepticism they save for the last viva — is covered end to end. The recurring questions with model answers close the appendix.

C.1 Language fundamentals (Experiments 1-2)

1. Structure of a C program: preprocessor directives, declarations, statements? — name each in Experiment 1's listing.
2. Keyword vs identifier vs user-defined name: is `PI` a keyword? Why is that the right answer?
3. Why is `int main(void)` better than `void main()`?
4. What does `-Wall -Wextra` change about a build? Why does the lab demand it?
5. Difference between `=`, `==`, and the fatal `if (x = 0)`.
6. Why is $5/2$ equal to 2 but $5.0f/2$ equal to 2.5 — and where in the experiments does that bite?
7. Order of the grade ladder: why not test `marks < 50` first? Give the input that breaks the wrong order.
8. What makes a switch faster than an if-ladder, and what makes a ladder more expressive than a switch?

C.2 Control flow & arithmetic (Experiments 3-5)

1. Fall-through: reproduce the bug in one sentence; name the two-line fix and the one-word style rule.
2. Why can switch not dispatch on `float` — and what does the standard permit as labels?
3. default clause: when may it be omitted? (One answer only: never in a program you didn't write.)
4. Why must the sine series use radians? What converges when you feed degrees?
5. State the recurrence $\text{term}_n = \text{term}_{n-1} \cdot (-x^2) / ((2n-2)(2n-1))$ — derive the divisor in one breath.
6. Why test `|term| < tol` rather than “`n = 10` times” for a fixed budget? When would the fixed budget be the better engineering choice?
7. For `D < 0` the program never calls `sqrt` on a negative number. Locate the two code lines that guarantee that.
8. What does the program print for `a = 0`, and why is that better than what C would compute without the guard?

C.3 Functions & pointers (Experiments 7-8, 13)

1. Prototype vs definition vs call — show one of each in Experiment 7.
2. Write a predicate without `if`: what expression replaces the body of `has_passed`?
3. Why is a predicate allowed to return `int` when it answers a yes/no question?
4. Give the three-statement temp swap; then give the two-statement version and show why it loses a value.
5. “C is always call by value.” Defend or attack the sentence using `commit_swap`.
6. When is passing `float *` cheaper than passing `float`? When is it cheaper to pass by value?
7. What is a wild pointer; what is a null pointer; which one crashes first?
8. What does `const float *` promise, and which call-site mistake does it turn into a compile error?

C.4 Arrays, strings & the algorithmic set (Experiments 6, 9-12)

1. Why does `scanf("%s")` fail on a course description? What function reads lines, and what extra byte does it keep?
2. Give the `strcspn` trim one-liner and say which character it hunts.
3. `strstr` returns a pointer. Give two facts you can extract from that pointer.
4. State the loop invariant of binary search and the two-line exit that proves absence.

5. Why does binary search need “ascending order” as a precondition rather than a hint?
6. After bubble pass *k*, which region is final, and what does the swapped flag detect? (Give the input that ends sorting in one pass.)
7. Tie handling: which character in the comparison preserves entry order for equal scores?
8. In units-rate, why is the inner loop over products the cache-friendly direction?
9. Write `my_strlen` and `my_strcat` from memory. Which one calls the other, and what does that cost per append?
10. Name two library string functions whose absence must be proved in Experiment 12 — how does the examiner check?

C.5 Derived datatypes & tools (Experiment 14, engineering practice)

1. What does a struct definition allocate? What does a struct variable allocate? Use the catalogue to answer.
2. Why can you copy a struct with `=` but not an array? Where does that asymmetry pay off in the sort extension?
3. Explain the `getchar()` after the count `scanf` as if the examiner has never seen the bug.
4. What is a translation unit? Which headers do the fourteen programs actually need?
5. Read this warning from a real build: “ignoring return value of `scanf`” — when does it matter?
6. What is the cost of a dangling pointer in an ATM that “temporarily swapped” balances and kept the temp address? (One sentence. The story is your own.)
7. Name the three TLP tools the syllabus suggests for this lab and one use of each in these experiments.

C.6 The six recurring questions — model answers

Question	Model answer (what a top record sounds like)
What is the difference between a compiler error and undefined behaviour?	An error stops the build; UB is a program that builds, runs, and answers with whatever the machine happened to leave behind. Writing <code>a[50]</code> in a 50-slot array is UB: it may print, it may crash — neither outcome is a reason to keep it.
Why does <code>scanf</code> need <code>&</code> but <code>printf</code> doesn't?	<code>scanf</code> must write into your variable, so it needs its address; <code>printf</code> only reads the value and a copy suffices. That one sentence is also the entire rationale of Experiment 8's pointers.
Array vs pointer — where are they the same, and where not?	An array name in an expression becomes the address of its first element (which is why array parameters decay and <code>sizeof</code> at the callee is a pointer size); an array object keeps its own size and cannot be assigned or copied whole — a pointer can be reassigned; a pointer to an array does not know the element count, which is why every function here passes (data, <i>n</i>).
How do you prove a program was tested, not just run?	The test matrix: the sample the question gives, the boundary of every rule (90/75/60/50, <code>low==high</code> , <code>i=n-1</code>), and an invalid input. In this lab the transcripts are the proof; your record should read the same way.
What does an algorithm gain you when the code is 20 lines?	The flowchart carries marks and catches design errors before they compile: Experiment 2's order bug and Experiment 14's newline bug are both visible on paper and invisible at a keyboard.
Why must the open-ended question “not be selected from Part-A”?	Because its rubric rewards design, not recall: the answer must contain a choice that could have gone another way (search strategy, sort choice, calling mechanism), and a justification in

Question	Model answer (what a top record sounds like)
	measured evidence — the defence, not the deliverable, earns the block.

Appendix D — References & Further Reading

D.1 Prescribed in the syllabus

1. Hassan Afyouni, Behrouz A. Forouzan. A Structured Programming Approach in C, 4th Edition, Cengage. [Textbook — theory paired to each experiment’s first section.]
2. Schildt, Herbert. C: The Complete Reference, 4th Edition, McGraw-Hill. [Reference — the syntax oracle for every A.x appendix table.]
3. Brian W. Kernighan and Dennis M. Ritchie. The C Programming Language, 2nd Edition, Prentice Hall of India. [Reference — the original, still the best second read on pointers and I/O.]

D.2 E-resources from the syllabus

1. Introduction to Programming in C — NPTEL online course: onlinecourses.nptel.ac.in/noc23_cs02/preview
2. C for Everyone: Programming Fundamentals — Coursera: coursera.org/learn/c-for-everyone
3. Computer Programming Virtual Lab (pointers experiment used alongside Experiment 8): cse02-iiith.vlabs.ac.in/exp/pointers/
4. C Programming: The Ultimate Way to Learn the Fundamentals of the C Language — pdfdrive.com/c-programming-the-ultimate-way-to-learn-the-fundamentals-of-the-clanguage-e187584209.html
5. C Programming: The Complete Reference (Schildt 4e preview) — viden.io/knowledge/programming-in-clanguage/attachment/28313/c-the-complete-reference-herbert-schildt-4th-edition-pdf/preview
6. Virtual labs portal for the TLP strategies: vlab.co.in

D.3 Tools used to produce these notes

- GCC 14.2.0 (Debian) with `-Wall -Wextra -std=c11`; GNU libc math library (`-lm`) on the link line for Experiments 1, 4, 5.
- Every transcript captured through a real pty terminal session so the echoed input appears exactly as a student at the lab bench would see it.
- Flowcharts drawn programmatically (SVG) with ANSI symbol conventions so the record can copy them by hand without redrawing logic.
- Book typography: DejaVu Serif / DejaVu Sans Mono, typeset via CSS Paged Media (WeasyPrint), mirroring the companion activity-book format of the same series.

D.4 For the curious

ISO/IEC 9899:2011 (C11) — the standard itself; sections 6.5.2.5 (pointers), 6.7.9 (initialisation), 7.21–7.24 (I/O and string library). The C FAQ (c-faq.com) for every “why does this work” question this lab generates. Effective C style checklists for the record’s inference sections.

About the Author

Dr. Lokesh M R is Professor in the Department of Information Science & Engineering at A J Institute of Engineering and Technology (AJIET), Mangaluru, and teaches first-year computing courses under the VTU 2022 scheme. His teaching practice centres on laboratory documentation that can be defended orally — the reason every experiment in this book pairs a worked program with a cap-

tured transcript and a viva trail. He is also the author of the companion activity notes Introduction to AI and Applications (1BAIA103/207) in the same series and format.

Corrections, suggestions and classroom reports are welcome at the department office. First Edition, September 2026 — programs verified against GCC 14.2 on the publication date.